GUJARAT TECHNOLOGICAL UNIVERSITY

Branch: MCA SEMESTER -2

Subject Name: Operating System

Subject Code: MC02094021

UNIT-1

INTRODUCTION:

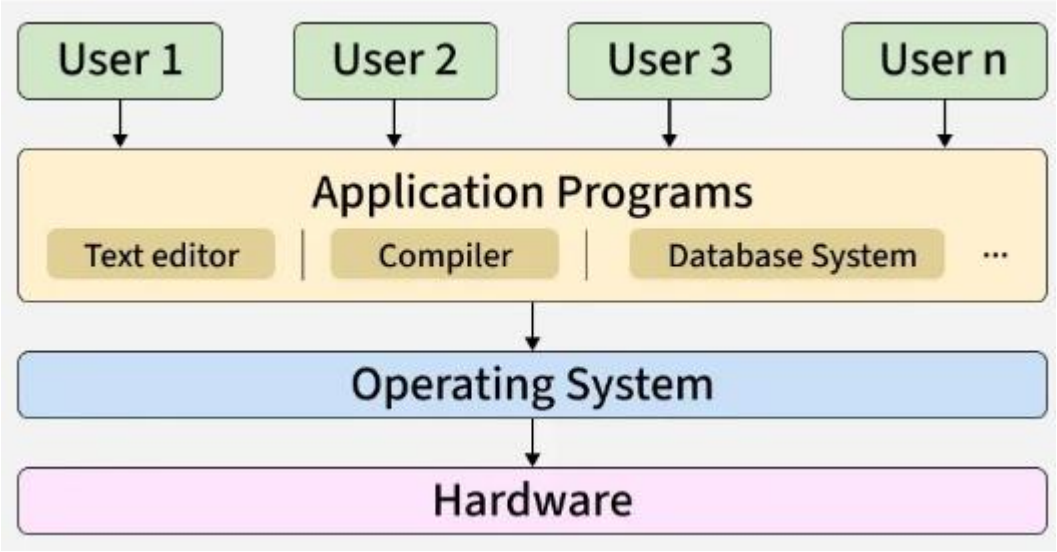
An operating system acts as an intermediary between the computer hardware and the user. In short, it is an interface between computer hardware and the user.

- The purpose of an operating system is to provide an environment in which a user can execute programs conveniently and efficiently.
- The operating system (OS) is a program that runs at all times on a computer. All other programs, including application programs, run on top of the operating system.
- It assigns resources such as memory, processors, and input/output devices to processes that need them. The assignment of resources has to be fair and secure.

Operating System and User Interaction

Every general-purpose computer consists of hardware, an operating system, system programs and application programs.

- The hardware includes CPU, ALU, memory, I/O devices, peripheral devices and storage devices.
- The system programs include compilers, loaders, editors and the OS itself.
- The application programs are user-level software that performs specific tasks.



An operating system is always running in the background to manage and coordinate the use of hardware resources among system and application programs for multiple users. The OS itself is not only a user interface. Instead, it provides an interface for user interaction, typically through:

- Command-Line Interface (CLI), (e.g., Bash, PowerShell)
- Graphical User Interface (GUI), (e.g., Windows desktop, macOS Finder)

Goals of Operating System

Primary Goals

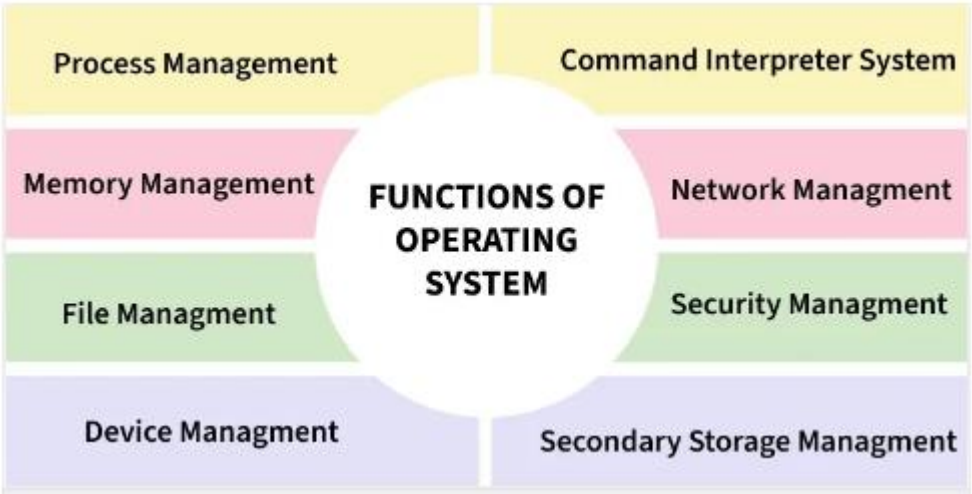
The primary goals of an operating system (OS) are to provide an easy to use and convenient environment for executing user programs.

- **User Convenience:** It should be easy to use, providing a user-friendly interface and making it simple to interact with the system.
- **Program Execution**: It facilitates the execution of user programs, providing the necessary environment and services for them to run**.**
- **Resource Management**: The OS manages and allocates the computer's resources, including the CPU, memory, disk storage and input/output devices, to ensure fair utilization.
- **Security**: The OS protects the system and user data from unauthorized access, ensuring confidentiality, integrity and availability of information.

Secondary Goals

- **Efficient Resource Utilization**: It should aim to maximize the performance and utilization of computer resources like CPU, Memory and I/O devices, ensuring that the system runs smoothly and efficiently.
- **Reliability**: It should be robust and reliable, able to handle errors and exceptions gracefully, ensuring that the system continues to operate smoothly. It should be modular in design and easy to debug.

Features of OS



Processor Management

- Keep tracks of the status of processes.
- The program to track the status is known as traffic controller.
- It allocates the CPU.
- And de-allocates the processor when it is not required.

Memory Management

- It keeps track of primary memory.
- Memory addresses that have already been allocated and the memory addresses of the memory that has not yet been used.
- In multiprogramming, the OS decides for how long the process must stay and the order in which processes are granted access to memory.
- It allocates the memory to a process when the process requests it and deallocates the memory when the process has terminated.

Device Management

- Keeping track of all devices connected to the system.
- Designates a program that is responsible for every device called the Input/output controller.
- It decides which process gets access to a device and for how long and allocates devices in an effective and efficient way.
- And de-allocates devices when they are not required.

File Management

Operating system also keeps track of the file system of a computer normally the file system is divided in directories which can be handled by the OS.

I/O Management

I/O management is one of the important tasks of the operating system. OS manages the IO devices like mouse, keyboard, printer, display, LED's etc.

I/O devices can be of two types −

- Synchronous I/O − Here, CPU waits for the I/O device
- Asynchronous I/O − Here, the CPU execution takes place parallel to the IO device.

Security Management

The confidential data stored in the system is protected by the operating system and does not allow the unauthorized person to access the system. It helps the system from malware attack and it acts as supervisor mode and gives the system a strongly protected firewall.

Network Management

Network management in operating systems involves controlling, configuring, and monitoring network resources to ensure optimal performance, security, and reliability. It manages data transmission, device communication, and traffic.

Applications of Operating System

- **Platform for Application Programs:** It provides a platform, on top of which, other programs, called application programs can run.
- **Managing Input-Output Unit:** It also allows the computer to manage its own resources such as memory, monitor, keyboard, printer, etc. Management of these resources is required for effective and fair utilization.
- **Multitasking:** It manages memory and allows multiple programs to run in their own space and even communicate with each other through shared memory.
- **Manages Memory and Files:** It manages the computer’s main memory and secondary storage. Additionally, it allows and deallocates memory to all tasks and applications.
- **Provides Security:** It helps to maintain the system and applications safe through the authorization process. Thus, the OS provides security to the system.

Advantages of Operating System

- Operating System manages external and internal devices for example, printers, scanners, and other.
- Operating System provides interfaces and drivers for proper communication between system and hardware devices.
- Allows multiple applications to run simultaneously.
- Manages the execution of processes, ensuring that the system remains responsive.
- Organizes and manages files on storage devices.
- Operating system allocates resources to various applications and ensures their efficient utilization**.**

Disadvantages of Operating System

- If an error occurred in your operating system, then there may be a chance that your data may not be recovered therefore always have a backup of your data.
- Threats and viruses can attack our operating system at any time, making it challenging for the OS to keep the system protected from these dangers.
- For learning about new operating system can be a time-consuming and challenging, Specially for those who using particular Operating system for example switching from Windows OS to Linux is difficult.
- Keeping an operating system up-to-date requires regular maintenance, which can be time-consuming.
- Operating systems consume system resources, including CPU, memory, and storage, which can affect the performance of other applications.

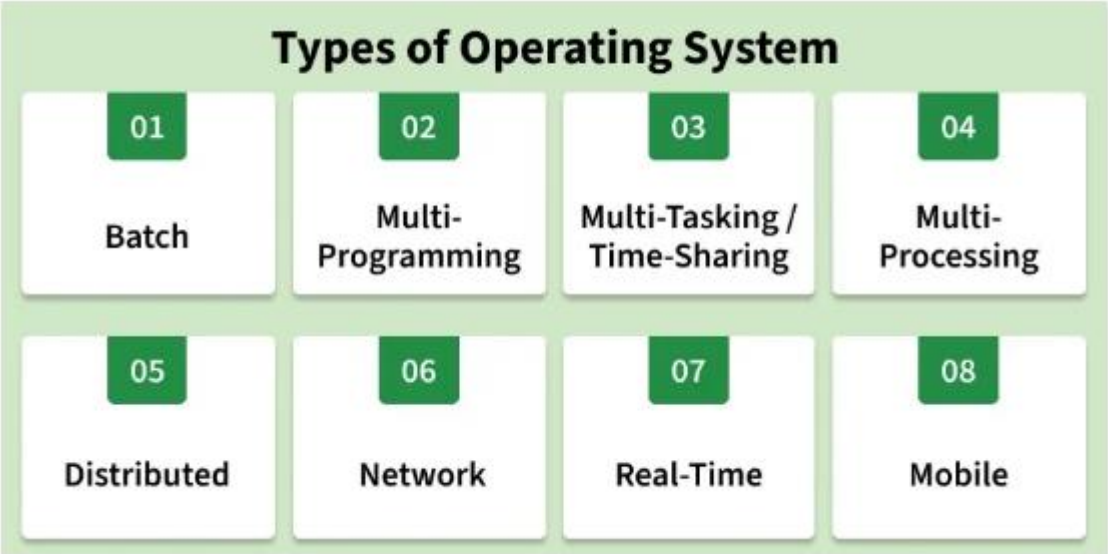
Generations of Operating systems

- **1940s-1950s: Early Beginnings**
    - Computers operated without operating systems (OS).
    - Programs were manually loaded and run, one at a time.
    - The first operating system was introduced in 1956. It was a batch processing system GM-NAA I/O (1956) that automated job handling.
- **1960s: Multiprogramming and Timesharing**
    - Introduction of multiprogramming to utilize CPU efficiently.

- Timesharing systems, like CTSS (1961) and Multics (1969), allowed multiple users to interact with a single system.

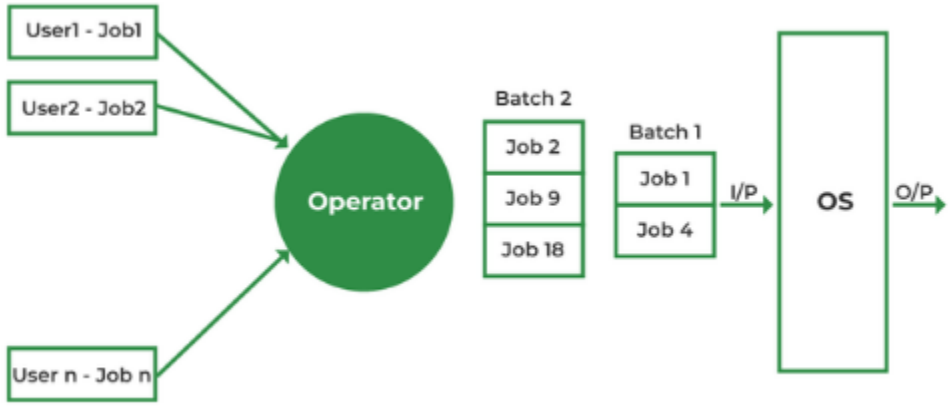
- **1970s: Unix and Personal Computers**
  - Unix (1971) revolutionized OS design with simplicity, portability, and multitasking.
  - Personal computers emerged, leading to simpler OSs like CP/M (1974) and PC-DOS (1981).
- **1980s: GUI and Networking**
  - Graphical User Interfaces (GUIs) gained popularity with systems like Apple Macintosh (1984) and Microsoft Windows (1985).
  - Networking features, like TCP/IP in Unix, became essential.
- **1990s: Linux and Advanced GUIs**
  - Linux (1991) introduced open-source development.
  - Windows and Mac OS refined GUIs and gained widespread adoption.
- **2000s-Present: Mobility and Cloud**
  - Mobile OSs like iOS (2007) and Android (2008) dominate.
  - Cloud-based and virtualization technologies reshape computing, with OSs like Windows Server and Linux driving innovation.

Types of Operating Systems



1. **Batch operating System**

A Batch Operating System is designed to handle large groups of similar jobs efficiently. It does not interact with the computer directly but instead processes jobs that are grouped by an operator. These jobs are queued and executed one after the other, without user interaction during the process.



Advantages

- **Minimal Idle Time:** The system minimizes idle time by processing jobs in a continuous sequence without human intervention.
- **Handling Repetitive Tasks:** Ideal for managing large, repetitive tasks, such as payroll and billing, with minimal effort.
- **Improved Throughput:** Batch systems can handle high volumes of jobs at once, improving overall system throughput.

Disadvantages

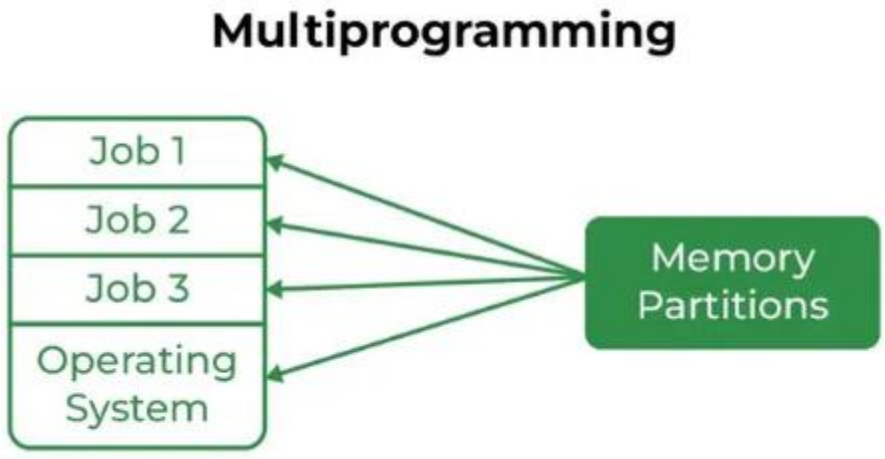
- **Inefficient CPU Utilization:** When a job is waiting for input/output (I/O), the CPU remains idle, leading to poor utilization of resources.
- **Increased Response Time:** The time between job submission and output can be high as all jobs are processed sequentially.
- **Lack of Real-Time Feedback:** Users cannot interact with the system in real-time, making it less suitable for interactive tasks.

Examples:

- *Ensurance Claim Processing*
- *Library Book Records*
- *Stock Market Reports*

2. **Multi-Programming Operating System**

In a Multi-Programming Operating System, multiple programs run in memory at the same time. The CPU switches between programs, utilizing its resources more effectively and improving overall system performance.



Advantages

- **Better CPU Utilization:** CPU stays busy by switching to another job during I/O wait.
- **Improved Throughput:** Multiple jobs run concurrently, increasing work done per unit time.
- **Efficient Resource Use:** CPU, memory, and I/O devices are shared effectively among processes.

Disadvantages

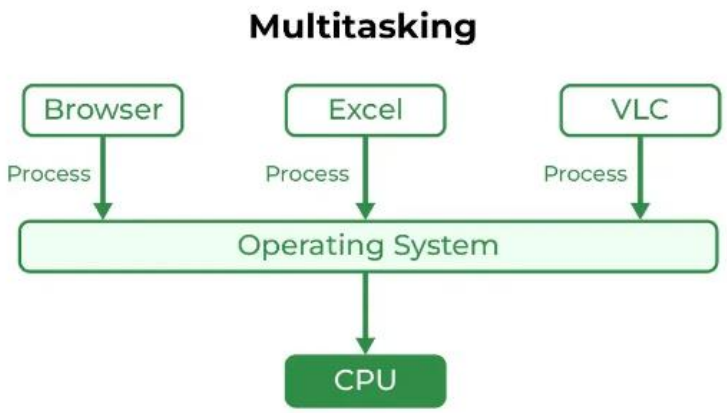
- **Complex Design:** Requires advanced memory management and CPU scheduling.
- Security Issues: More programs in memory increase chances of unauthorized access.
- High Memory Requirement: Needs larger RAM to run multiple programs together.

Examples:

- *Banking systems*
- *Railway servers*
- *Billing machines*

3. Multi-tasking/Time-sharing Operating systems

Multitasking OS is a type of multiprogramming system where each process runs in a round-robin manner. Every task gets a fixed time slice called a quantum. After the quantum ends, the OS switches to the next task, allowing multiple tasks—whether from one user or many—to run smoothly on a single system.



Advantages

- **Equal CPU Access:** Each task gets a fair share of CPU time.
- **Reduced Software Duplication:** Many users can run the same software without needing separate copies.
- **Low CPU Idle Time:** Efficient scheduling keeps the CPU busy.

Disadvantages

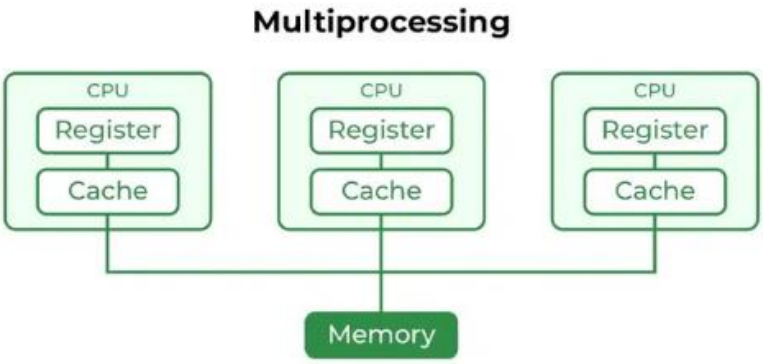
- **Lower Reliability:** System failures affect all users.
- **Security Concerns:** Multiple users increase risks to data integrity and privacy.
- **Communication Issues:** Data sharing between users can cause conflicts.

Examples:

- *IBM VM/CMS*
- *TSO (Time Sharing Option*
- *Windows Terminal Services*

4. Multi-Processing Operating System

Multi-Processing OS is a type of Operating System in which more than one CPU is used for the execution of resources. It betters the throughput of the System.



Advantages

- **Faster Processing:** Multiple CPUs work simultaneously, increasing overall system speed.
- **High Reliability:** If one processor fails, others can continue working (fault tolerance).
- **Supports Heavy Tasks:** Ideal for computation-intensive applications like scientific or industrial tasks.

Disadvantages

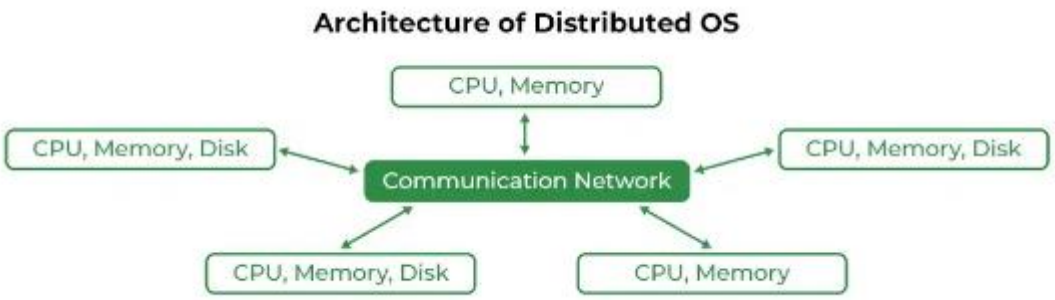
- **High Cost:** Multiple processors and complex hardware increase system cost.
- **Complex Design:** Requires advanced OS support for communication and task distribution.
- **Not Always Efficient:** Poor task distribution can lead to idle processors and wasted resources.

Examples:

- *UNIX*
- *Linux (Ubuntu, Red Hat, Debian)*
- *macOS*

5. Distributed Operating System

Distributed operating systems connect multiple independent computers through a shared communication network. Each system has its own CPU and memory but works together as a single unit. The main benefit is remote access, allowing users to use files and software stored on other connected systems.



Advantages

- **Independent Systems:** Failure of one machine does not affect others.
- **Easily Scalable:** New systems can be added to the network easily.
- **Lower Processing Delays:** Tasks are handled faster across multiple machines.

Disadvantages

- **Network Dependency:** If the main network fails, communication stops.
- **Lack of Standardization:** No well-defined language or model for building such systems.
- **High Cost & Complexity:** Hardware is expensive, and the software is complex and not widely understood.

Issues With Distributed Operating System

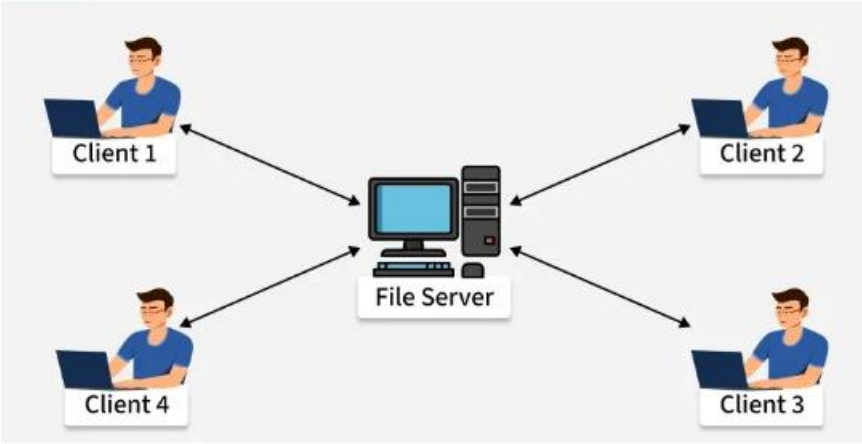
- Networking causes delays in the transfer of data between nodes of a distributed system. Such delays may lead to an inconsistent view of data located in different nodes and make it difficult to know the chronological order in which events occurred in the system.
- Control functions like scheduling, resource allocation and deadlock detection have to be performed in several nodes to achieve computation speedup and provide reliable operation when computers or networking components fail.
- Messages exchanged by processes present in different nodes may travel over public networks and pass through computer systems that are not controlled by the distributed operating system. An intruder may exploit this feature to tamper with messages or create fake messages to fool the authentication procedure and masquerade as a user of the system.

Examples:

- *LOCUS*
- *MICROS*
- *Amoeba*

6. Network Operating System

A Network Operating System (NOS) runs on a server and manages data, users, security, applications, and other network functions. It allows shared access to files, printers, and resources within a small private network. Users can see the configuration and connections of other users, which is why these systems are considered tightly coupled systems.



Advantages

- **Centralized and Stable Servers:** Provide reliable management of resources.
- **Easy Upgrades:** New hardware and technologies can be added without difficulty.
- **Remote Access:** Users can access the server from different locations and devices.

Disadvantages

- **High Server Cost:** Setting up and maintaining servers is expensive.
- **Dependency on Server:** Most operations rely on a central server.
- **Regular Maintenance Needed:** Frequent updates and technical support are required.

Examples:

- *Microsoft Windows Server 2003*
- *UNIX, Linux*
- *Mac OS X*

7. Real-Time Operating System

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called **response time. Real-time systems** are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Types of Real-Time Operating Systems

- **Hard Real-Time Operating System:** Used where strict timing is essential and any delay is unacceptable, such as airbags or automatic parachutes. These systems avoid virtual memory to ensure immediate response.
- **Soft Real-Time Operating System**: Used where timing is important but minor delays are acceptable. These systems aim to give quick and predictable responses but do not require perfect accuracy. They are commonly used in multimedia applications, gaming, video streaming, and other interactive tasks.

Advantages

- **Maximum Consumption:** Maximum utilization of devices and systems, thus more output from all the resources.
- **Error-Free,:** These types of systems are error-free.
- **Memory Allocation:** Memory allocation is best managed in these types of systems.

Disadvantages

- **Limited Tasks:** Very few tasks run at the same time and their concentration is very less on a few applications to avoid errors.
- **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write.
- **Thread Priority:** It is not good to set thread priority, as these systems are much less prone to switching tasks.

Examples:

- *Scientific experiments*
- *Medical imaging systems*
- *Robots*

8. Mobile Operating Systems

Mobile operating systems are designed specifically for mobile devices such as smart phones and tablets. Examples of such operating systems are Android and iOS. These operating systems manage the hardware and software resources of the device, providing a platform for running applications and ensuring a seamless user experience.

Advantages

- **User-Friendly Interfaces**: Mobile operating systems are designed to be intuitive and easy to use, making them accessible to a wide range of users.
- **Extensive App Ecosystems**: The availability of a vast number of applications allows users to customize their devices to meet their specific needs.
- **Connectivity Options**: Mobile operating systems support multiple connectivity options, enabling users to stay connected wherever they go.

Disadvantages

- **Battery Life Constraints**: Despite advancements in power management, battery life remains a challenge for mobile devices, especially with heavy usage.
- **Security Risks**: Mobile devices are susceptible to various security threats, such as malware and phishing attacks, which can compromise user data.
- **Fragmentation**: In the case of Android, the wide range of devices and customizations can lead to fragmentation, making it difficult for developers to ensure compatibility across all devices.

Examples:

- *Android*
- *iOS*
- *Blackberry*

Operating System Services

- The OS coordinates the use of hardware and application programs for users.
- It provides a platform for running application software.

- It is a set of special programs that help the computer function properly.
- It controls input–output devices and system operations.
- It manages program execution and file handling.

1. Program Execution

The Operating System manages program execution by loading programs into memory and scheduling them using algorithms like FCFS, SJF. It ensures smooth execution of user and system programs while preventing deadlocks where processes wait indefinitely. The OS also allocates system resources efficiently for proper functioning.

- Loads programs into memory for execution.
- Uses CPU scheduling algorithms (FCFS, SJF, etc.) to decide execution order.
- Handles and prevents deadlocks during execution.
- Manages system resources for efficient overall functioning.

2. Input Output Operations

The Operating System manages all input-output operations and enables communication between users, programs, and device drivers. Device Device drivers act as the software interface for hardware, allowing the OS to control devices smoothly. The OS also provides programs with safe and timely access to input-output devices whenever required.

- Manages all input-output operations.
- Communicates with hardware through device drivers.
- Ensures proper coordination between devices and the system.
- Provides programs controlled access to input-output devices.

3. Communication Between Processes

The Operating System manages communication between processes and enables smooth data transfer among them. It handles both local communication on the same computer and communication across networked systems. This ensures efficient and coordinated exchange of information.

- Manages inter-process communication (IPC).
- Enables data transfer between processes.
- Supports communication across networked computers.
- Ensures secure and efficient information exchange.

4. File Management

The Operating System manages all file operations by granting programs the required access permissions such as read-only or read-write. It allows users to create, delete, store, and organize files across devices like, floppy disk or pen drives. The OS also decides how data should be stored and manipulated for efficient file management.

- Grants programs file access with permissions like read-only or read-write.
- Allows users to create, delete, and organize files.
- Manages file storage across devices such as hard disks and pen drives.
- Decides how data is stored and manipulated for efficient file management.

5. Memory Management

The OS manages memory like a cricket team manager choosing players. Just as the manager decides whether a player enters the playing 11 or 15 based on performance, the OS checks whether a new program meets the requirements for memory. If it does, the OS decides how much memory the program needs and loads it into an appropriate location, preventing unnecessary memory usage.

- OS checks whether a new program meets the requirements to get memory.
- It decides how much memory the program needs.
- Loads the program into a suitable memory location.
- Prevents programs from using unnecessary or extra memory.

6. Process Management

Process management can be understood by imagining the kitchen stove as the CPU and the chef as the Operating System. The chef must cook multiple dishes (programs) and ensures no single dish takes too much time so that every dish gets a fair chance to be cooked. In this way, the OS schedules and manages all programs efficiently to keep the system running smoothly.

- CPU is like the kitchen stove where execution happens.
- OS (chef) ensures each program gets fair execution time.
- Prevents any program from occupying the CPU for too long.
- Schedules all programs efficiently to keep the system running smoothly.

7. Security and Privacy

Security

The Operating System keeps the computer safe from unauthorized users by adding protective security layers. It defends the system from threats like viruses and hackers through tools such as firewalls and antivirus software. These measures ensure the safety of the computer and its data.

- Blocks unauthorized access to the system.
- Protects against viruses and malware.
- Uses firewalls and antivirus tools for defense.
- Ensures overall system and data safety.

Privacy

The Operating System helps keep personal information private, similar to locking a door that only you can open. It ensures that sensitive files and data remain hidden from others. In this way, the OS respects user confidentiality and protects important information.

- Keeps personal data hidden from others.
- Provides access control options like passwords.
- Ensures only authorized users can view files.
- Protects sensitive information from misuse.

8. Resource Management

The Operating System manages how system resources are shared among different processes. It allocates CPU time using CPU Scheduling Algorithms, handles memory management, and controls input-output devices. By deciding which process gets which resource, the OS ensures proper and efficient use of all available resources.

- Shares system resources among multiple processes.
- Allocates CPU time using scheduling algorithms.
- Manages memory and input-output devices.

- Ensures efficient and proper use of all resources.

9. User Interface

The Operating System provides a user interface that allows users to interact with the system either through a command-line interface(CLI) or a Graphical User Interface (GUI). In a CLI, the command interpreter executes user-typed commands. A GUI offers a mouse-based window and menu system that makes interaction easier and more visual.

- Provides both CLI and GUI for user interaction.
- CLI uses a command interpreter to run user commands.
- GUI offers windows, icons, and menus for easy use.
- Helps users communicate smoothly with the operating system.

10. Networking

This service enables communication between devices on a network by managing connections and controlling data exchange. It allows devices to connect to the internet, send and receive data packets, and maintain stable communication. Through proper network management, the system ensures smooth and reliable data transfer.

- Manages network connections between devices.
- Supports internet access and communication.
- Sends and receives data packets reliably.
- Ensures smooth and stable network operations.

11. Error Handling

The Operating System detects and handles errors occurring in the CPU, memory, or input-output devices to keep the system stable. It prevents frequent errors, avoids deadlocks, and fixes issues that appear during program execution. A secure OS also acts as a protective layer against external threats or breaches.

- Detects and handles CPU, memory, and I/O errors.
- Prevents frequent errors and avoids deadlocks.
- Fixes issues that occur during task execution.
- Provides security measures against external threats and breaches.

12. Time Management

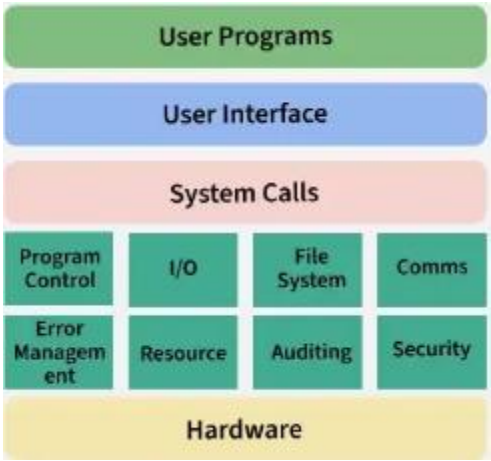
Imagine a traffic light as the Operating System, guiding cars (programs) on when to stop, wait, or move. Red represents the simple queue, yellow is the ready queue, and green means the program is under execution. By switching lights at proper intervals, the OS ensures all programs move smoothly without traffic or delays.

- Red light = programs waiting in the simple queue.
- Yellow light = programs in the ready queue.
- Green light = programs under execution.
- OS changes states at proper intervals to ensure smooth program flow.

System Calls

User programs cannot directly access hardware or critical OS resources because it would make the system unstable and insecure. To maintain safety, the operating system provides system calls — controlled interfaces that allow user programs to request services from the kernel. These calls act as a gateway between user mode and kernel mode. System Calls are,

- A way for programs to interact with the operating system.
- Provide the services of the operating system to the user programs.
- Only entry points into the kernel are executed in kernel mode.



Example:

- Opening a file in C (fopen) internally uses system calls like open().
- Running a program in Linux uses fork() and exec() system calls.
- Printing on screen uses the write() system call.

How do System Calls Work?

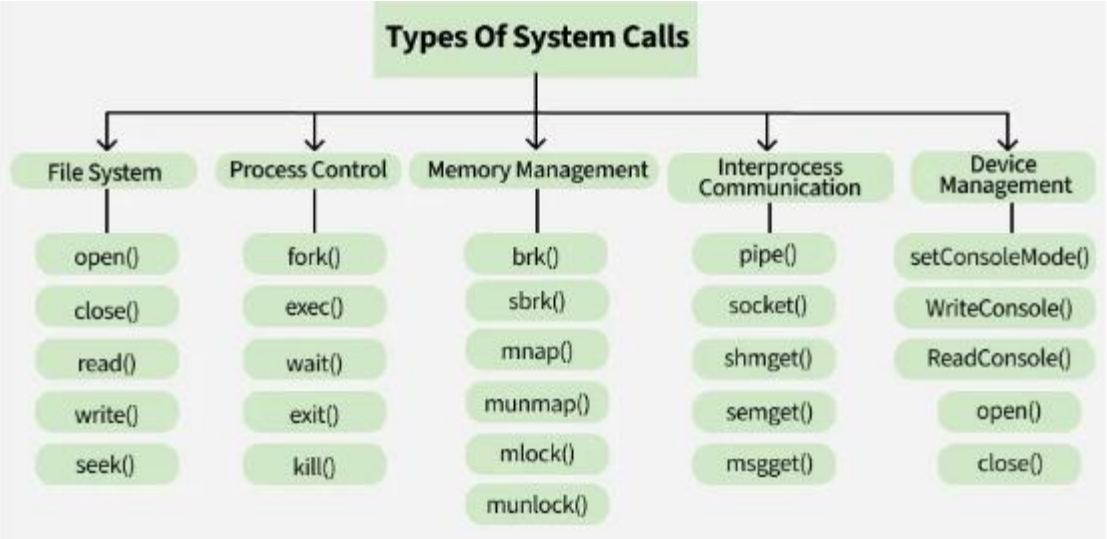
A system call is a controlled entry point that allows a user program to request a service from the operating system. Here's how it works:

- The user program executes a system call instruction (e.g., using syscall or int 0x80).
- The CPU switches from user mode → kernel mode for safe execution.
- The kernel identifies the system call number and performs the requested operation (file access, process creation, memory allocation, etc.).
- After completing the task, the kernel switches back to user mode.
- The result (success/failure/data) is returned to the program.
- Without system calls, every program would need its own way to access hardware, leading to inconsistent and insecure systems.

System calls do not always cause context switching. They primarily involve a mode switch from user mode to kernel mode. A context switch happens only when the calling process is blocked, not during every system call.

Types of System Calls

Services provided by an OS are typically related to any kind of operation that a user program can perform like creation, termination, forking, moving, communication, etc. Similar types of operations are grouped into one single system call category. System calls are classified into the following categories:



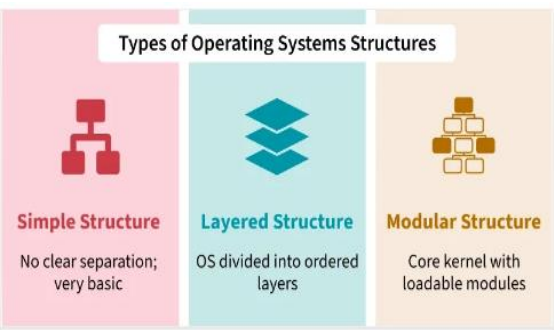
- **File System**: Used to create, open, read, write, and manage files and directories.
- **Process Control**: Used to create, execute, synchronize, and terminate processes.
- **Memory Management**: Used to allocate, deallocate, and manage memory for processes.

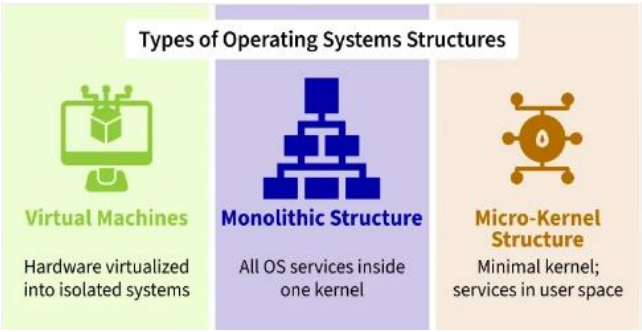
- **Interprocess Communication (IPC)**: Used for data exchange and communication between different processes.
- **Device Management**: Used to request and release devices, and to perform read/write operations on them.

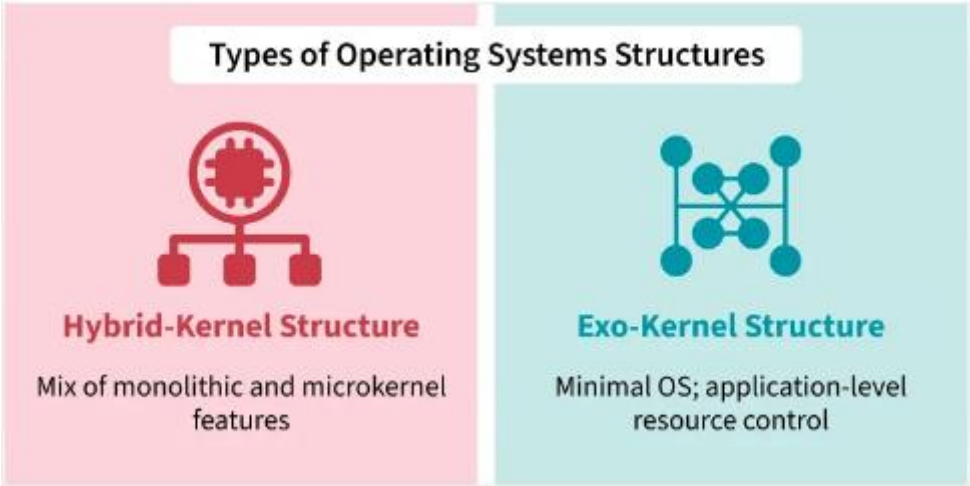
Structure of an OS-Layered, Monolithic, Microkernel Operating Systems

An operating system can be built using different structures depending on how its components are organized and how they interact inside the kernel. Its system structure acts as a blueprint that shows how the OS is arranged so it can be understood and adapted to different needs. Since OS design is complex, breaking it into smaller, manageable parts makes development easier. The structure defines the strategy used to integrate these components within the kernel.

- OS structure depends on component organization and interaction.
- Acts as a blueprint that guides OS design.
- Breaking the OS into smaller parts simplifies development.
- Structure defines how components are integrated into the kernel.



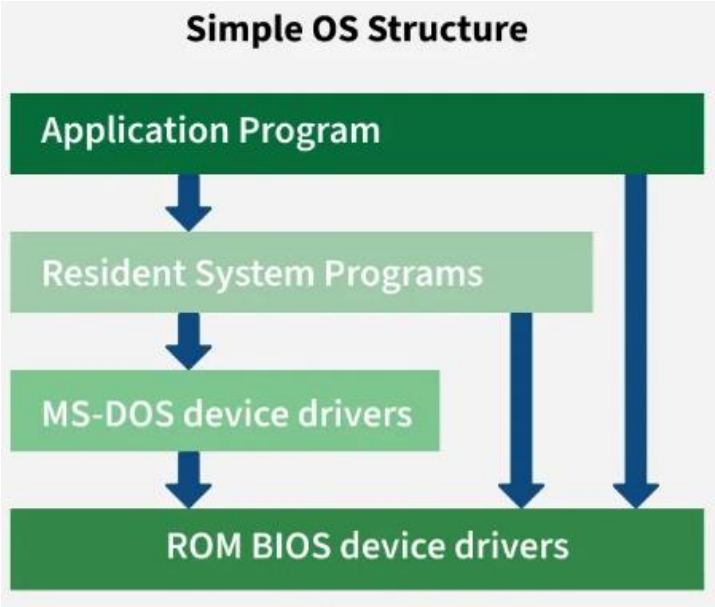




Simple Structure

Simple-structure operating systems have no well-defined architecture and are small, simple and limited in functionality. Their interfaces and functional levels are not clearly separated, which makes them less reliable. Because of this lack of separation, a failure in any user program can cause the entire system to crash.

- No well-defined structure; small and limited.
- Interfaces and functionality levels are not separated.
- MS-DOS is an example of this structure.
- Applications can directly access basic I/O routines.
- A user program failure can crash the entire system.

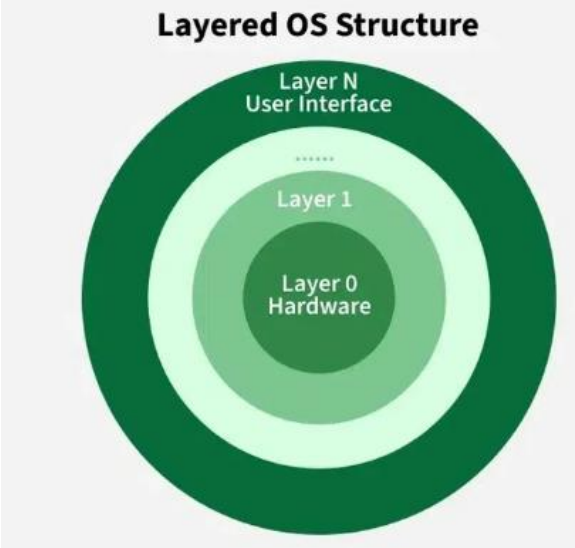


Layered Structure

In Layered structure, the OS is divided into levels where each layer uses the services of the one below it—hardware at the bottom and the user interface at the top. This design improves organization and simplifies debugging, as errors can be isolated to specific layers. However, it adds overhead because data must pass through multiple layers and requires careful planning. UNIX is an example of this structure.

- OS is divided into multiple layers (hardware at bottom, UI at top).
- Each layer uses the functions of the layer below it.
- Easier debugging because errors are isolated to specific layers.
- Adds overhead due to data passing through many layers.
- Requires careful planning; layers can only use lower layers.

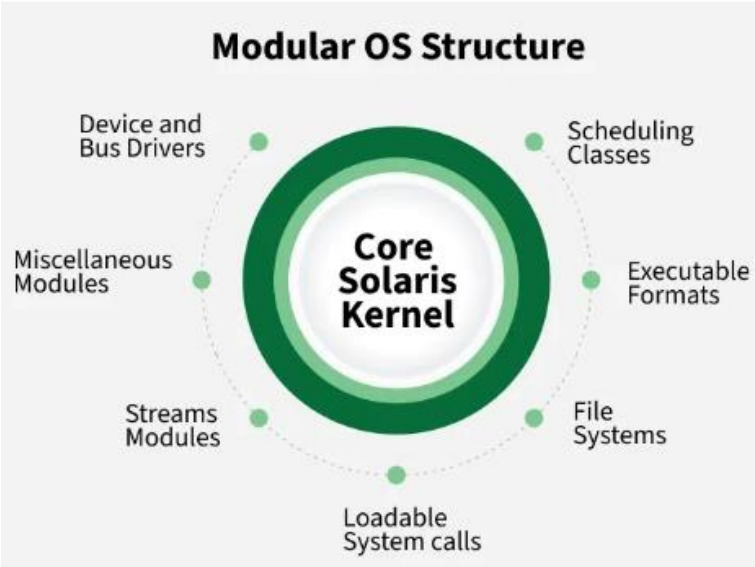
- UNIX is an example of a layered structure.



Modular Structure

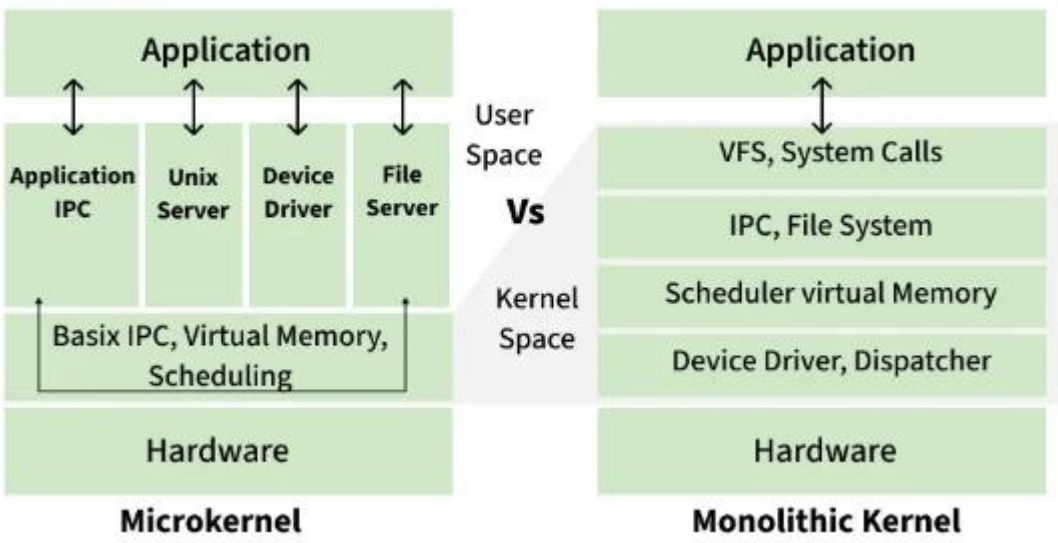
A modular structure uses a kernel that contains only essential core components, while additional services are added as loadable modules during boot or runtime. It is similar to a layered design because it has defined and protected interfaces, but it is more flexible since modules can communicate freely. This approach is widely used in modern systems such as Solaris.

- Kernel contains only core components.
- Extra services are added as dynamically loadable modules.
- More flexible than layered structure; modules can call any other module.
- Used in systems like Solaris.



Microkernel vs. Monolithic Kernel

Monolithic Kernel, all OS services run in kernel space, enabling high performance but risking stability, whereas a Microkernel runs only essential services in kernel space and others in user space, enhancing modularity and security at the cost of some performance due to inter-process communication overhead.



Micro-Kernel

A microkernel organizes the operating system by keeping only the most essential functions inside the kernel. These include basic process management, communication between system components, and minimal hardware control. All other services are placed outside the kernel in separate user-space programs.

- Components such as device drivers, file systems, and network services run as independent processes in user space.
- This approach keeps the kernel small, reduces the chance of system-wide failures, and makes the operating system more secure and easier to maintain.

Advantages of Micro-Kernel

- Size of Micro-Kernel is smaller, so it is easy to use.
- Easy to extend Micro-Kernel
- Easy to port Micro-Kernel
- Micro-Kernel is less prone to errors and bugs. One such example of this is Mac OS.

Disadvantages of Micro-Kernel

- The execution of Micro-Kernel is slower.
- Only the most important services are present inside the kernel and rest of the operating systems are present inside system application program.

- The communication between client process & services running in user address space is established through message that further reduces the speed of execution.

Monolithic Kernel

A monolithic kernel design places all operating system components—both user services and kernel-level services—into the same address space, running entirely in kernel mode. This arrangement allows different parts of the system to interact quickly and efficiently without additional communication overhead.

- This architecture includes major functions such as memory management, process control, device drivers, and file systems within one large kernel.
- While this setup can improve performance, it also increases complexity, as a problem in any single component can affect the entire system and make maintenance more challenging.

Advantages of Monolithic Kernel

- Monolithic Kernel is an all in one piece where user services & kernel services are implemented under the same address space.
- It has a faster speed of execution.
- One such example of this is Linux. It is simple to design and has a very high performance.

Disadvantages of Monolithic Kernel

- The monolithic kernel has a larger size since both user services & kernel services are implemented under the same space.
- Since it is larger in size, it becomes hard to extend the Monolithic Kernel.
- Hard to export and port the monolithic kernel
- Unlike micro kernel, Monolithic kernel is more prone to errors and bugs. Thus, the system encounters more errors that the usual being.

Microkernel vs Monolithic Kernel

Microkernel	Monolithic kernel
User services and kernel services are placed in separate address spaces.	User and kernel services share the same address space.

Microkernel	Monolithic kernel
More complex to design.	Easier to design and implement.
Smaller in size.	Larger compared to microkernel.
New functionalities can be added more easily.	Adding new functionalities is more difficult.
Requires more code to design.	Requires less code than a microkernel.
Failure of one component does not affect the entire system.	Failure of one component can cause the whole system to fail.
Slower execution speed.	Faster execution speed.
Easier to extend.	Not easy to extend.
IPC uses messaging queues.	IPC uses signals and sockets.
Easier to debug.	More difficult to debug.
Simple to maintain.	Requires extra time and resources for maintenance.
Requires message forwarding and context switching.	Does not require message passing or context switching during normal operations.

Microkernel	Monolithic kernel
Kernel provides only IPC and low-level device management. **Example:** macOS.	Kernel includes all major OS services. **Example:** Windows 95.

Concept of Virtual Machine:

A Virtual Machine (VM) is a software-based computer that runs within a physical computer (known as the host). It emulates the behavior of a physical machine and can run its own operating system (guest OS) and applications independently just like a real computer.

For example, you can run a Linux VM inside a Windows PC using tools like VirtualBox or VMware.

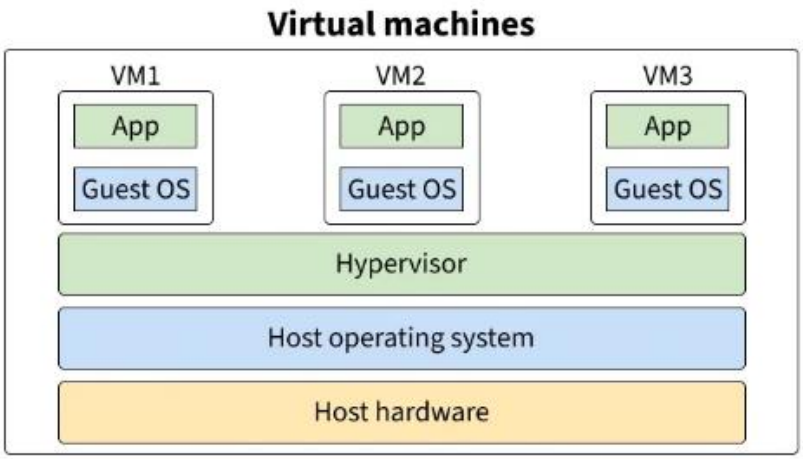
Working of Virtual Machines

A VM uses virtualization technology to abstract the hardware of a physical machine (CPU, memory, storage, etc.) into a virtual environment. This is managed by a special software called a hypervisor, which allocates system resources to multiple VMs.

Each VM behaves like a separate computer, even though it's running on shared hardware. It includes:

- Virtual CPU
- Virtual memory
- Virtual disk (storage)
- Network interface card (NIC)

Example: When you run multiple processes on a regular OS, they appear to have separate CPUs and memory (thanks to CPU scheduling and virtual memory). A VM extends this illusion to the OS level itself.



Types of Virtual Machines

There are **two main types** of VMs:

1. Process Virtual Machine

- Designed to run a single application (or process).
- Example: Java Virtual Machine (JVM), which allows Java programs to run on any platform.

2. System Virtual Machine

- Emulates an entire physical machine.
- Can run a full OS like Linux, Windows, or macOS.
- Example: VMs created using VirtualBox, VMware, or Hyper-V.

How to Set Up a Virtual Machine

Setting up a VM involves the following steps (example for installing a Linux VM):

1. Create a new virtual machine
2. Allocate virtual disk space
3. Attach a virtual network adapter
4. Install the guest operating system (Linux)
5. Register and activate the VM (if needed)
6. Install necessary drivers or guest additions

Most platforms like VirtualBox or VMware offer wizards to simplify this setup.

Uses of Virtual Machines

- Running multiple OSs on a single machine
- Testing new software or OS safely (sandboxed)
- Running legacy applications
- Server virtualization (hosting multiple VMs on a single server)
- Secure development and debugging (isolate risky tasks)

Hypervisor

A hypervisor is software (sometimes firmware or hardware-based) that enables virtualization by separating physical resources from VMs.

There are **two types**:

- **Type 1 (Bare Metal)**: Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V)
- **Type 2 (Hosted)**: Runs on top of a host OS (e.g., VirtualBox, VMware Workstation)

The hypervisor is responsible for assigning CPU, memory, disk, and network to each VM.

Advantages of Virtual Machines

- **Isolation**: Each VM is fully independent and secure from others.
- **Flexibility**: Run different OSs on one device.
- **Cost & energy saving**: Multiple VMs can run on a single machine.
- **Disaster recovery**: Easy to back up, clone, and restore.
- **Legacy support**: Run old apps on modern hardware.
- **Simplified testing**: Perfect for software development and security testing.

Types of Virtualization (Beyond VM)

Virtualization is not limited to just virtual machines. Here are 5 major types:

1. Hardware Virtualization (Server Virtualization)

Hardware virtualization is the process of creating multiple virtual machines (VMs) that run on a single physical server using a **hypervisor**.

- A hypervisor (like VMware ESXi or Microsoft Hyper-V) directly interacts with the physical hardware.
- It allocates physical resources (CPU, memory, storage) to each VM.
- Each VM operates independently with its own OS and applications.

Benefits:

- Efficient hardware utilization.
- Cost savings (fewer physical servers needed).
- Scalability and easier deployment of services.
- Fault isolation: if one VM crashes, others remain unaffected.

2. Software Virtualization

Software virtualization allows an operating system or application to run in an environment that is different from its native platform.

- A software layer mimics the required hardware or operating system.
- Example: Running Android OS on a Windows PC using an emulator like BlueStacks.

Benefits:

- Cross-platform compatibility.
- Testing apps on different OS versions.
- Reduced software conflicts

3. Storage Virtualization

Storage virtualization combines multiple physical storage devices into a single **logical storage pool**, making management easier and more flexible.

- Software abstracts the physical storage from multiple devices.
- The system presents the unified storage as a single resource to users and applications.

Benefits:

- Centralized storage management.
- Better performance through load balancing.
- Easier scaling and disaster recovery.

4. Network Virtualization

Network virtualization abstracts network resources, allowing multiple virtual networks to be created on a single physical infrastructure.

- Combines hardware (switches, routers) and software resources into a virtual network.
- Divides bandwidth into independent channels for better resource allocation.

Benefits:

- Improved network efficiency and security.
- Easier management and scaling.
- Dynamic provisioning of network resources.

5. Desktop Virtualization

Desktop virtualization separates the desktop environment from the physical machine and stores it on a remote server.

- Users access their virtual desktop over a network using any device (PC, tablet, thin client).
- All processing and data remain on the central server.

Benefits:

- Centralized control and security.
- Easier software updates and patches.
- Flexibility for remote work.

PROCESSES, THREAD & PROCESS SCHEDULING

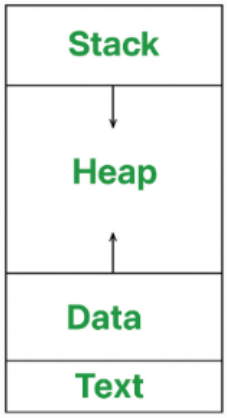
Processes

A process is a program in execution. For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

- A process is an 'active' entity instead of a program, which is considered a 'passive' entity.
- A single program can create many processes when run multiple times; for example, when we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created). .

How Does a Process Look Like in Memory?

A process in memory is divided into several distinct sections, each serving a different purpose. Here's how a process typically looks in memory.



- **Text Section:** A text or code segment contains executable instructions. It is typically a read only section
- **Stack:** The stack contains temporary data, such as function parameters, returns addresses, and local variables.
- **Data Section:** Contains the global variable.
- **Heap Section:** Dynamically memory allocated to process during its run time.

Attributes of a Process

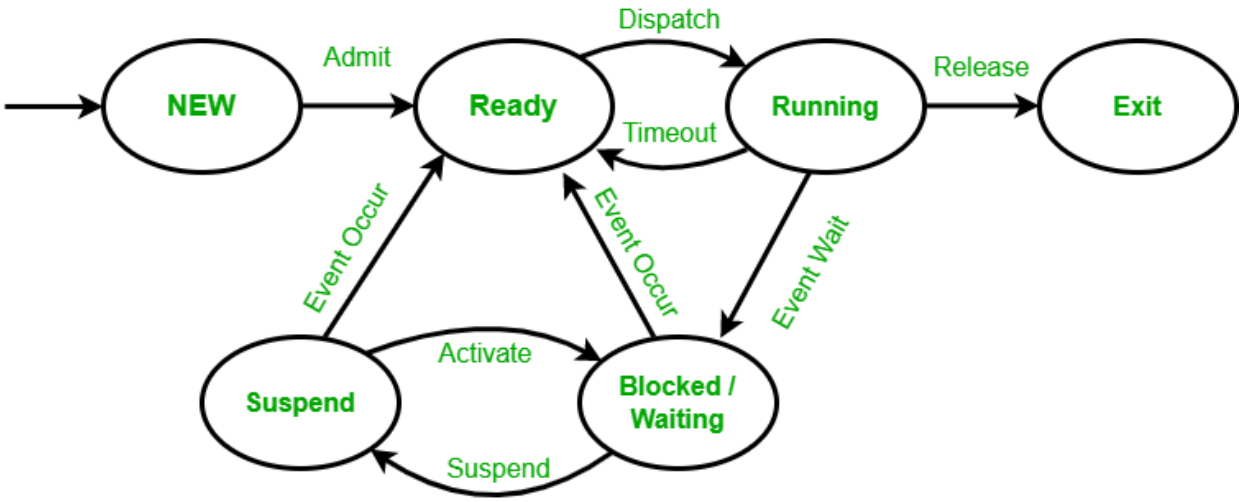
A process has several important attributes that help the operating system manage and control it. These attributes are stored in a structure called the Process Control Block (PCB) (sometimes called a task control block). The PCB keeps all the key information about the process, including:

1. **Process ID (PID):** A unique number assigned to each process so the operating system can identify it.
2. **Process State:** This shows the current status of the process, like whether it is running, waiting, or ready to execute.
3. **Priority and other CPU Scheduling Information:** Data that helps the operating system decide which process should run next, like priority levels and pointers to scheduling queues.
4. **I/O Information:** Information about input/output devices the process is using.
5. **File Descriptors:** Information about open files and network connections.
6. **Accounting Information:** Tracks how long the process has run, the amount of CPU time used, and other resource usage data.
7. **Memory Management Information**: Details about the memory space allocated to the process, including where it is loaded in memory and the structure of its memory layout (stack, heap, etc.).

Different states of a Process

A process in an operating system passes through multiple states as it begins execution, waits for resources, gets scheduled, runs, and eventually finishes. These stages collectively describe the complete lifecycle of a process.

The process state process lifecycle expands the basic two-state idea by separating processes that are simply waiting for CPU time from those that cannot run because they are waiting for an external event. This makes the model more accurate for real operating systems, where processes may pause for input, data, or device responses before continuing.

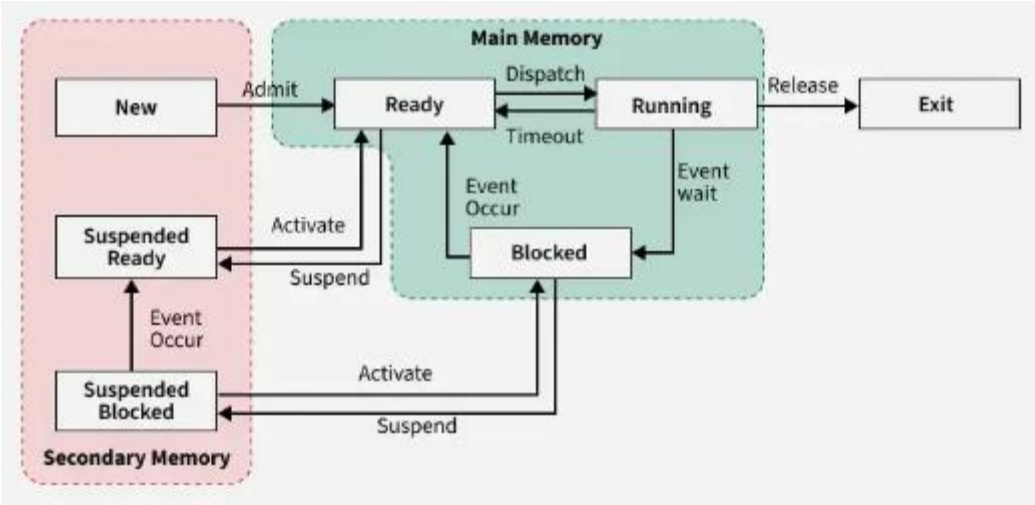


The five states are:

- **New:** The process has just been created. It hasn't started running yet, but its PCB (Process Control Block) is prepared.
- **Ready:** The process is loaded into memory and prepared to run as soon as the CPU becomes free.
- **Running:** The CPU is currently executing this process. With a single CPU system, only one process can be in this state at a time.
- **Blocked/Waiting:** The process cannot continue execution because it is waiting for an event like I/O completion or data input.
- **Exit/Terminate:** The process has completed its execution or has been stopped, and the operating system removes it from memory.

Process State transitions

A process moves through different states as it is created, prepared for execution, runs on the CPU, waits for resources, or completes. These states help the operating system manage how processes use memory, CPU, and I/O devices.



Seven State

The main process states are:

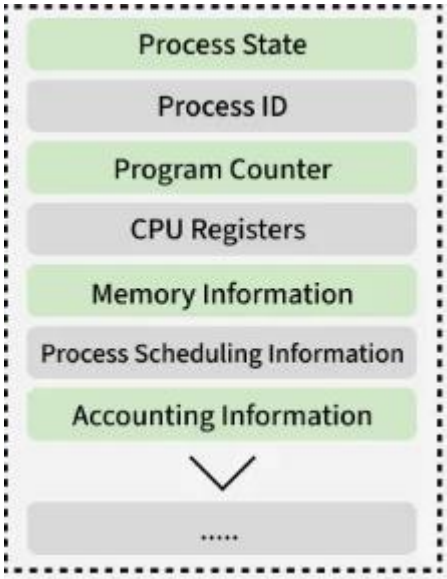
- **New:** The process is in the creation phase. The program exists in secondary memory, and the OS prepares its PCB before loading it into main memory.
- **Ready:** The process has been loaded into main memory and is prepared to run. It is waiting in the ready queue until the CPU becomes available.
- **Running:** The CPU is currently executing the instructions of the process. Only one process can be in this state at a time on a single-processor system.
- **Blocked/Waiting:** The process cannot continue because it is waiting for an event such as I/O completion, user input, or access to a locked resource. Once the event is complete, it moves back to the ready state.
- **Terminated:** The process has completed its execution or has been stopped. The OS deletes its PCB and frees all resources allocated to it.
- **Suspend Ready:** A ready process that has been swapped out of main memory and placed in secondary storage due to memory shortage. When brought back to main memory, it returns to the ready state.
- **CPU-Bound / I/O-Bound:** A CPU-bound process spends most of its time performing computations, while an I/O-bound process spends more time waiting for input/output operations.

Process Control Block (PCB)

A Process Control Block (PCB) is a data structure used by the operating system to keep track of process information and manage execution. It helps the OS monitor and control process execution

- Each process is given a unique Process ID (PID) for identification.

- The PCB stores details such as process state, program counter, stack pointer, open files, and scheduling info.
- During a state transition, the OS updates the PCB with the latest execution data.
- It also includes register values, CPU quantum, and process priority.
- The Process Table is an array of PCBs that maintains information for all active processes.



Terminologies used in Process Control Block

- **Process state:** Stores whether the process is running, waiting, ready, or terminated
- **Process number Or PID:** Every process is assigned a unique id known as process ID or PID.
- **Program counter:** Program Counter stores the address of the next instruction that is to be executed for the process.
- **Register:** Registers in the PCB, it is a data structure. When a processes is running and it's time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values is read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.
- **Memory limits:** This field contains the information about memory management system used by the operating system. This may include page tables, segment tables, etc.
- **List of Open files:** This information includes the list of files opened for a process.

Applications

- Helps the operating system schedule processes and allocate CPU resources efficiently.

- Enables efficient resource utilization and sharing by providing detailed resource-usage information.
- Supports context switching through stored CPU registers and stack pointer information.
- Assists in process synchronization by keeping track of waiting states and required resources.
- Tracks process states and resource usage to determine which process should be executed next.

Context switching

Context switching is the process where the CPU stops running one process, saves its current state, and loads the saved state of another process so that multiple processes can share the CPU effectively.

Need in Multitasking

It is essential in multitasking systems where many processes need CPU time.

- In multitasking, the CPU keeps switching between processes.
- This makes it seem like processes are running at the same time, even though the CPU works on one process at a time.
- Without context switching, one process could monopolize the CPU, and others would have to wait indefinitely

Role in Scheduling

- The scheduler decides which process should run next based on a scheduling algorithm (like Round Robin, Priority Scheduling, etc.).
- Context switching executes that decision by stopping the current process and starting the chosen one.
- The dispatcher is the component that actually performs the switch.

Context Switching Triggers

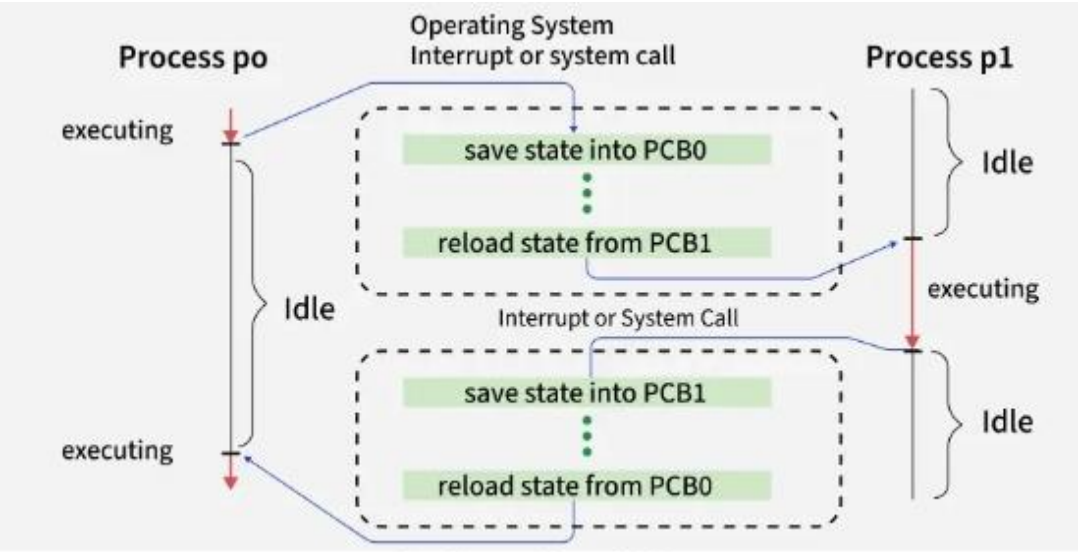
Context Switching Happen:

- When a high-priority process comes to a ready state (i.e. with higher priority than the running process).
- An Interrupt occurs.
- User and kernel-mode switch (It is not necessary though)

- Preemptive CPU scheduling is used.

When switching between user mode and kernel/user mode is necessary, operating systems use the kernel/user switch.

Working Process of Context Switching



State Diagram of Context Switching

Explanation of the Above Diagram

The diagram illustrates how the CPU alternates execution between two processes — p0 and p1 — through context switching.

1. Execution Phase (Process p0)

- Process p0 is running while p1 remains idle.
- At some point, an interrupt or system call occurs (e.g., a timer interrupt or I/O completion).

2. Saving Current State (p0 → PCB0)

- The operating system pauses p0.
- The current state of p0 (register values, program counter, etc.) is saved into its Process Control Block (PCB0).
- This ensures that p0 can later resume exactly where it left off.

3. Loading New Process State (PCB1 → p1)

- The OS retrieves the saved state of p1 from PCB1.
- This step restores p1's CPU context so it can continue execution from its last saved point.

4. Execution Phase (Process p1)

- Now, p1 is running while p0 is idle.
- Another interrupt or system call occurs, triggering another switch.

5. Saving State of p1 (p1 → PCB1): The OS saves the current state of p1 into PCB1.

6. Reloading State of p0 (PCB0 → p0)

- The saved state of p0 from PCB0 is loaded back into the CPU registers.
- p0 resumes execution right where it left off.

Overhead

- Time spent switching is called context switch overhead.
- During this time, the CPU is not performing actual process work it's just preparing to run the next process.
- If context switching happens too often, CPU efficiency drops.

Factors Affecting Switch Time

Context-switch times are highly dependent on hardware support. For example, some processors (such as the Sun Ultra (SPARC) provide multiple sets of registers. In this case, a context switch simply requires changing the pointer to the current register set. However, if there are more active processes than available register sets, the system resorts to copying register data to and from memory, as before. Additionally, the more complex the operating system, the greater the amount of work that must be done during a context switch.

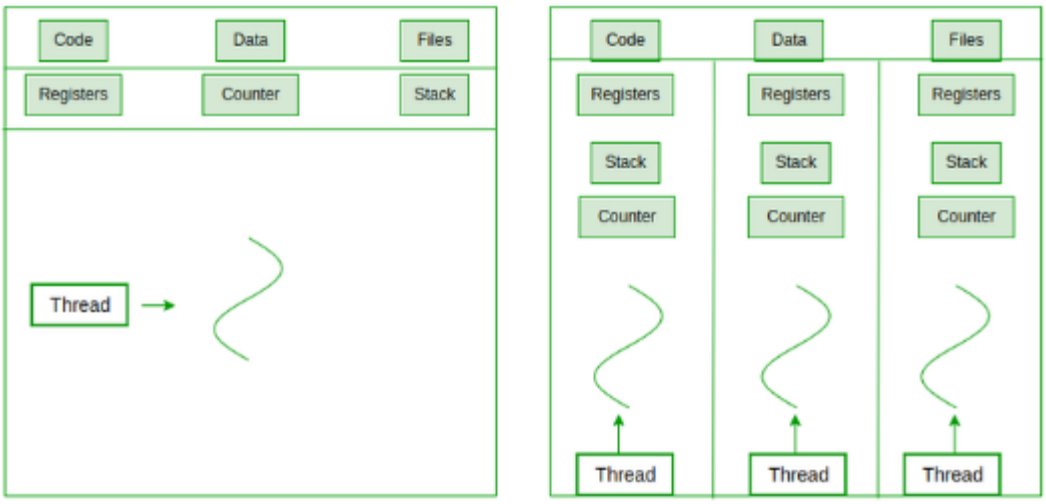
- Hardware support for fast register saving/loading.
- Number of registers (more registers = more time to save/load).
- Memory speed for PCB storage.
- Efficiency of OS kernel code.

Thread

A thread is a single sequence stream within a process and is called a lightweight process because it is smaller and faster. It allows multiple tasks to run simultaneously, improving program efficiency.

- In single-core systems, it creates an illusion of parallelism; in multi-core systems, threads can execute truly in parallel across different cores.
- Each thread has its own program counter, register set, and stack space.

- Threads share code, data, and operating system resources within the same process.



Single Threaded Process Multi Threaded Process

Why Do We Need Threads

Threads are needed in modern operating systems and applications because they:

- **Improve Performance:** Threads allow multiple tasks to run at the same time (parallel or interleaved), making programs execute faster.
- **Increase Responsiveness:** If one thread is busy, another can handle user actions, keeping the application responsive.
- **Enable Concurrency:** Multiple operations like saving files, processing data, and handling user input can happen simultaneously.
- **Better CPU Utilization:** On multi-core systems, threads can run on different cores, improving overall system performance.
- **Efficient Resource Sharing:** Threads share the same memory and resources within a process, making communication faster and reducing overhead.

Components of Threads

These are the basic components of the Operating System.

- **Stack Space:** Stores local variables, function calls, and return addresses specific to the thread.
- **Register Set:** Hold temporary data and intermediate results for the thread's execution.
- **Program Counter:** Tracks the current instruction being executed by the thread.

Advantages of Threads

1. **Better CPU Utilization**

- Multiple threads can run concurrently, keeping the CPU busy and improving system performance.

2. **Faster Execution**
    - Tasks can be divided into smaller subtasks and executed simultaneously, reducing overall execution time.
3. **Improved Responsiveness**
    - Applications (like web browsers or games) remain responsive because one thread can continue working while another waits for input/output.
4. **Resource Sharing**
    - Threads within the same process share memory, files, and resources easily.
5. **Lower Creation Cost**
    - Creating a thread is faster and requires fewer resources than creating a new process.
6. **Efficient Communication**
    - Communication between threads is faster since they share the same address space.

Disadvantages of Threads

1. **Complex Programming**
    - Writing multithreaded programs is more difficult than single-threaded programs.
2. **Synchronization Problems**
    - Multiple threads accessing shared data can cause **race conditions** and require synchronization mechanisms (locks, semaphores).
3. **Debugging Difficulty**
    - Errors like **deadlocks**, **race conditions**, and **thread starvation** are hard to detect and debug.
4. **Security Issues**
    - Since threads share memory, one thread can accidentally or intentionally modify another thread's data.
5. **Context Switching Overhead**

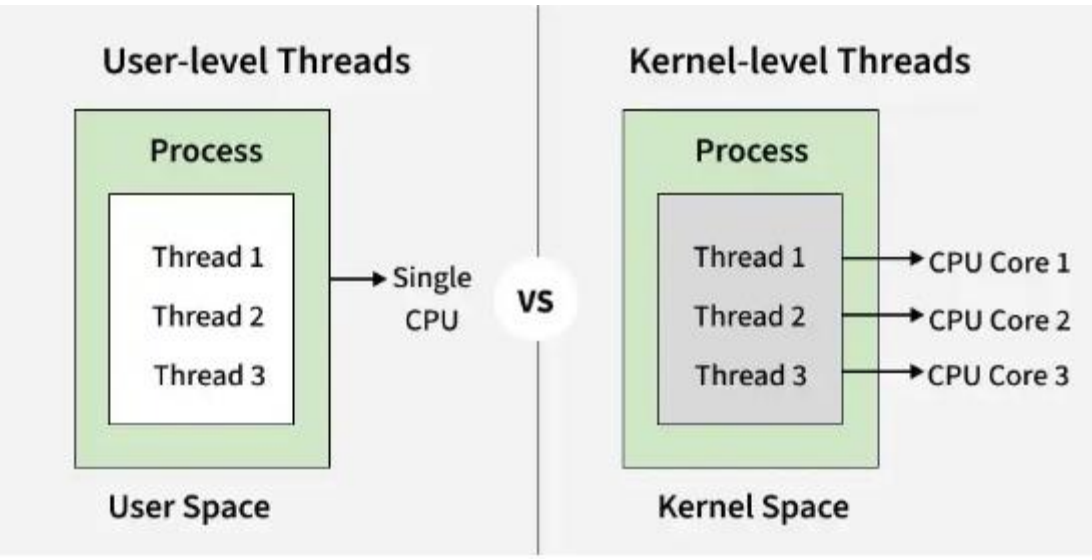
- Switching between threads still consumes CPU time and can reduce efficiency if too frequent.

6. **Crash Impact**
    - If one thread crashes in a process, it may affect the entire process.

Types of Threads

Threads are mainly classified based on how they are managed and scheduled in an operating system. There are two primary types of threads.

- User Level Thread
- Kernel Level Thread



User-Level Threads (ULTs)

- Managed entirely in user space using a thread library; the kernel is unaware of them.
- Switching between ULTs is fast since only program counter, registers, and stack need to be saved/restored.
- Do not require system calls for creation or management, making them lightweight.
- If one thread makes a blocking system call, the entire process (all threads) is blocked.
- Scheduling is done by the application itself, which may not be as efficient as kernel-level scheduling.
- Cannot fully utilize multiprocessor systems because the kernel schedules processes, not individual user-level threads.

Kernel-Level Threads (KLTs)

- Managed directly by the operating system kernel; each thread has an entry in the kernel's thread table.
- The kernel schedules each thread independently, allowing true parallel execution on multiple CPUs/cores.
- Handles blocking system calls efficiently; if one thread blocks, the kernel can run another thread from the same process.
- Provides better load balancing across processors since the kernel controls all threads.
- Context switching is slower compared to ULTs because it requires switching between user mode and kernel mode.
- Implementation is more complex and requires frequent interaction with the kernel.
- Large numbers of threads may add extra load on the kernel scheduler, potentially affecting performance.

Process vs Thread

Process	**Thread**
Program in execution	Part of a process
Takes more time to create & terminate	Takes less time to create & terminate
Context switching is slow	Context switching is fast
Heavyweight	Lightweight
Less efficient communication	More efficient communication
Blocking one process doesn't affect others	Blocking a user-level thread may block all

Process	Thread
Uses system calls	Created using APIs (may not need OS call)
Has its own PCB, stack, address space	Shares PCB & address space, has own TCB & stack
Does not share data	Shares data with other threads

User-Level Thread Vs Kernel-Level Thread

User-Level Thread (ULT)	Kernel-Level Thread (KLT)
Implemented by user-level libraries	Implemented by the Operating System
Not recognized by the OS	Recognized by the OS
Fast context switching with less overhead	Slower context switching with more overhead
Blocking blocks the entire process	Only the blocked thread is affected
Limited use of multiprocessing	Fully utilizes multiprocessing
Fast and simple creation and management	Slower and more complex management
Threads share the same address space	Each thread has its own address space

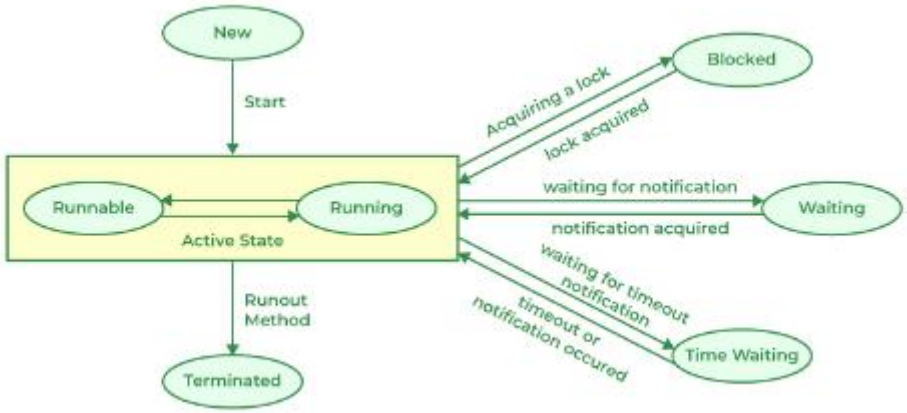
User-Level Thread (ULT)	Kernel-Level Thread (KLT)
More portable, works on any OS	OS-dependent and less portable

Various states of thread in os

A thread in Java can exist in any one of the following states at any given time. A thread lies only in one of the shown states at any instant:

1. New State
2. Runnable State
3. Blocked State
4. Waiting State
5. Timed Waiting State
6. Terminated State

The diagram below represents various states of a thread at any instant:



Life Cycle of a Thread

There are multiple states of the thread in a lifecycle as mentioned below:

1. **New Thread:** When a new thread is created, it is in the **new state**. The thread has not yet started to run when the thread is in this state. When a thread lies in the new state, its code is yet to be run and has not started to execute.

2. **Runnable State**: A thread that is **ready to run** is moved to a runnable state. In this state, a thread might actually be running or it might be ready to run at any instant of time. It is the responsibility of the thread scheduler to give the thread, time to run. A multi-threaded program allocates a fixed amount of time to each individual thread. Each and every thread get a small amount of time to run. After running for a while, a thread pauses and gives up the CPU so that other threads can run.

3. **Blocked:** The thread will be in blocked state **when it is trying to acquire a lock** but currently the lock is acquired by the other thread. The thread will move from the blocked state to runnable state when it acquires the lock.

4. **Waiting state**: The thread will be in waiting state **when it calls wait()** method or **join()** method. It will move to the runnable state when other thread will notify or that thread will be terminated.

5. **Timed Waiting**: A thread lies in a timed waiting state when it **calls a method with a time-out parameter**. A thread lies in this state until the timeout is completed or until a notification is received. For example, when a thread calls sleep or a conditional wait, it is moved to a timed waiting state.

6. **Terminated State:** A thread terminates because of either of the following reasons:

    - Because it exits normally. This happens when the code of the thread has been entirely executed by the program.
    - Because there occurred some unusual erroneous event, like a segmentation fault or an unhandled exception.

Concept of multithreads

Multithreading is a technique where a process is divided into smaller execution units called threads that run concurrently.

- A thread is also called a lightweight process. Concurrency or Parallelism within a process is achieved by dividing a process into multiple threads.
- Multithreading improves system performance and responsiveness by allowing multiple threads to share CPU, memory and I/O resources of a single process.
- Example: In a browser, each tab can be a thread. In MS Word, one thread formats text while another processes inputs.

How are User Threads Mapped with Kernel Threads?

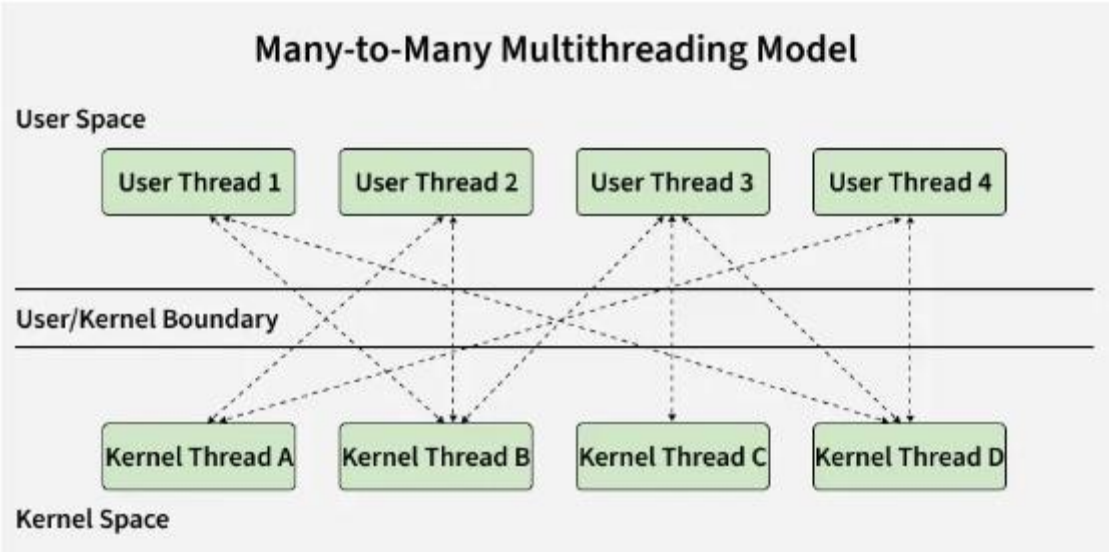
Multithreading model are of three types.

- Many to many model.

- Many to one model.
- One to one model.

Many to Many Model

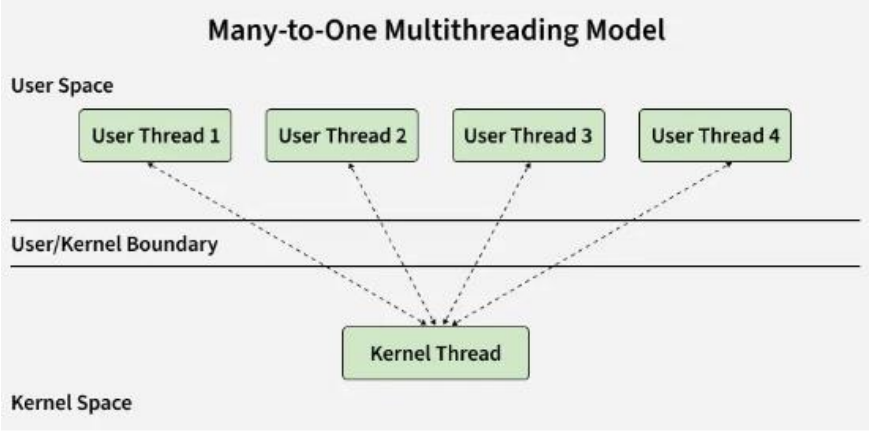
In this model, we have multiple user threads multiplex to same or lesser number of kernel level threads. Number of kernel level threads are specific to the machine, advantage of this model is if a user thread is blocked we can schedule others user thread to other kernel thread. Thus, System doesn't block if a particular thread is blocked.
It is the best multi threading model.



Many to One Model

In this model, we have multiple user threads mapped to one kernel thread. In this model when a user thread makes a blocking system call entire process blocks. As we have only one kernel thread and only one user thread can access kernel at a time, so multiple threads are not able access multiprocessor at the same time.

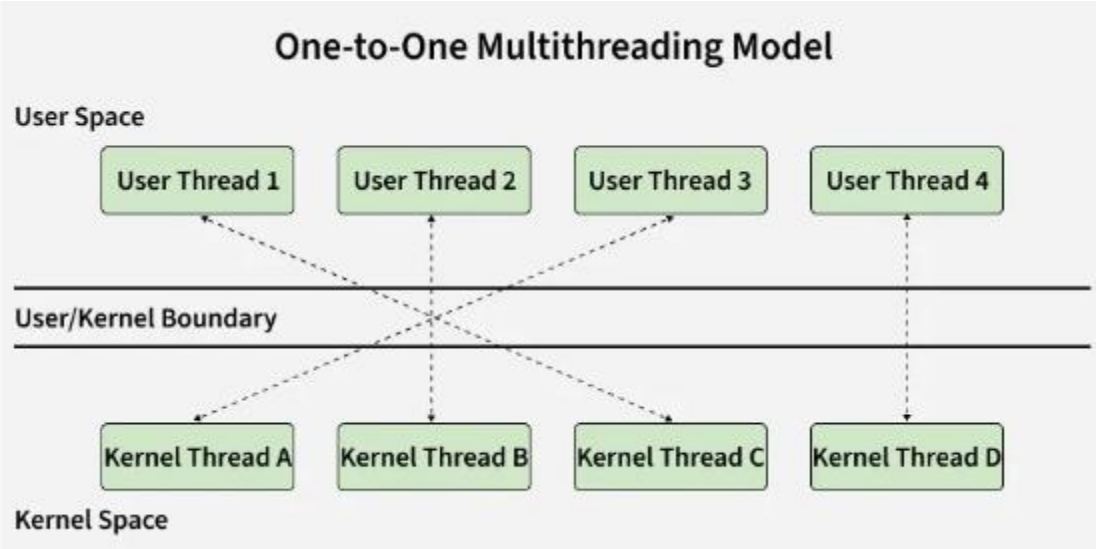
The thread management is done on the user level so it is more efficient.



One to One Model

In this model, one to one relationship between kernel and user thread. In this model multiple thread can run on multiple processor. Problem with this model is that creating a user thread requires the corresponding kernel thread.

As each user thread is connected to different kernel , if any user thread makes a blocking system call, the other user threads won't be blocked.



Is Multithreading Possible Without OS Support?

- Multithreading can be done without OS support, as seen in Java's model.
- Threads are managed by the Java Virtual Machine (JVM), not the OS.
- The application manages thread creation, scheduling, and context switching using a thread library.
- The OS is unaware of these threads and treats the process as single-threaded.

Applications

1. Transaction Processing

- Widely used in banking, online payments, and mobile recharges.
- Multiple transactions can be processed simultaneously without delays.

2. Web and Internet Applications

- Threads handle multiple client requests at the same time (e.g., web servers, chat servers).
- Improves responsiveness and efficiency of online platforms.

3. Banking & Financial Systems

- Online fund transfers, balance updates, and background verification tasks run in parallel.
- Prevents bottlenecks during high traffic (e.g., during salary credits or sales).

4. Telecom & Recharge Services

- Multiple recharge or service requests are processed concurrently.
- Ensures quick response and smooth functioning for millions of users.

Process Scheduling:

Process Scheduling is the method used by the Operating System (OS) to decide which process will use the CPU at a particular time. Since many processes are in memory but only one CPU is available, the OS schedules processes to use the CPU efficiently.

Objectives of Process Scheduling

1. Maximum CPU Utilization – Keep the CPU busy as much as possible.
2. Maximum Throughput – Complete more processes in less time.
3. Minimum Waiting Time – Reduce the time processes wait in the ready queue.
4. Minimum Turnaround Time – Reduce total time taken to complete a process.
5. Minimum Response Time – Provide quick response to users.

Process Schedulers

Process Schedulers are fundamental components of operating systems responsible for deciding the order in which processes are executed by the CPU. In simpler terms, they manage how the CPU allocates its time among multiple tasks or processes that are competing for its attention.

1. Long-Term Scheduler (Job Scheduler)

The Long-Term Scheduler is responsible for loading processes from disk into main memory so they can begin execution.

- Transfers processes from the Job Queue to the Ready Queue.
- Controls the degree of Multi-programming — the number of processes present in memory or ready state at any time.
- Carefully selects a balanced mix of I/O-bound and CPU-bound processes to ensure efficient system performance.
- Helps avoid a situation where either the CPU or I/O devices remain idle.
- In many modern time-sharing systems (such as Windows), a long-term scheduler may not exist; new processes are directly admitted to memory for short-term scheduling.

- It is the **slowest** among all schedulers, as it operates less frequently.

2. Short-Term Scheduler (CPU Scheduler)

The Short-Term Scheduler (STS) is responsible for selecting a process from the ready queue and assigning the CPU to it.

- Frequently selects the next process to execute from the ready state.
- Ensures no process suffers from starvation.
- Uses various CPU scheduling algorithms to decide process order.
- Maximizes CPU utilization by keeping the processor as busy as possible.
- Calls the dispatcher, which performs the actual context switch.
- It is the **fastest** scheduler, since it operates very frequently (often every few milliseconds).
- Saving context (process control block) of previously running process if not finished.
- Switching system mode to user mode.
- Jumping to the proper location in the newly loaded program.

3. Medium-Term Scheduler

The Medium-Term Scheduler (MTS) manages swapping, which temporarily moves processes between main memory and disk.

- Swaps processes out of memory when they are waiting (e.g., blocked for I/O) to reduce the degree of multiprogramming.
- Frees memory for other active processes.
- Swaps processes back into memory when they are ready to continue execution, allowing them to resume from where they left off.
- Helps maintain an effective mix of CPU-bound and I/O-bound processes.
- It operates faster than the long-term scheduler but slower than the short-term scheduler.

Some Other Schedulers

I/O Schedulers

- Manage order of read/write operations.
- Reduce I/O wait time and improve performance.

- Schedule I/O requests using algorithms like FCFS, RR, SCAN, C-SCAN.

Real-Time Schedulers

- Used in systems where tasks must meet deadlines.
- Ensure timely execution of critical tasks.
- Use algorithms like EDF and RM.

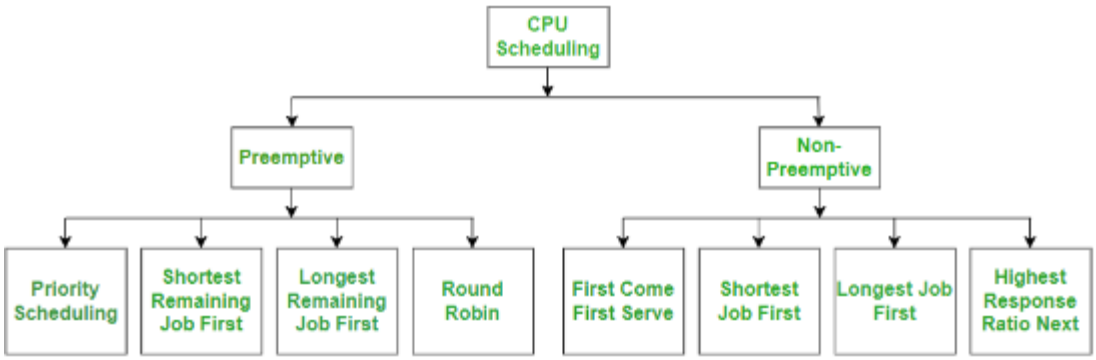
Scheduling objectives

- **CPU Utilization:** The main purpose of any CPU algorithm is to keep the CPU as busy as possible. Theoretically, CPU usage can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the system load.
- **Throughput:** The average CPU performance is the number of processes performed and completed during each unit. This is called throughput. The output may vary depending on the length or duration of the processes.
- **Turn Round Time:** For a particular process, the important conditions are how long it takes to perform that process. The time elapsed from the time of process delivery to the time of completion is known as the conversion time. Conversion time is the amount of time spent waiting for memory access, waiting in line, using CPU and waiting for I/O.
- **Waiting Time:** The Scheduling algorithm does not affect the time required to complete the process once it has started performing. It only affects the waiting time of the process i.e. the time spent in the waiting process in the ready queue.
- **Response Time:** In a collaborative system, turnaround time is not the best option. The process may produce something early and continue to computing the new results while the previous results are released to the user. Therefore another method is the time taken in the submission of the application process until the first response is issued. This measure is called response time.

Different Types of CPU Scheduling Algorithms

There are mainly two types of scheduling methods:

- **Preemptive Scheduling:** Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to the ready state.
- **Non-Preemptive Scheduling:** Non-Preemptive scheduling is used when a process terminates , or when a process switches from running state to waiting state.



FCFS

First Come, First Serve (FCFS) is one of the simplest types of CPU scheduling algorithms. It is exactly what it sounds like: processes are attended to in the order in which they arrive in the ready queue, much like customers lining up at a grocery store.

FCFS Scheduling is a non-preemptive algorithm, meaning once a process starts running, it cannot be stopped until it voluntarily relinquishes the CPU, typically when it terminates or performs I/O. This method schedules processes in the order they arrive, without considering priority or other factors.

How Does FCFS Work?

The mechanics of FCFS are straightforward:

1. **Arrival:** Processes enter the system and are placed in a queue in the order they arrive.
2. **Execution:** The CPU takes the first process from the front of the queue, executes it until it is complete, and then removes it from the queue.
3. **Repeat:** The CPU takes the next process in the queue and repeats the execution process.

Example of FCFS CPU Scheduling:

To understand the First Come, First Served (FCFS) scheduling algorithm effectively, we'll use two examples -

- one where all processes arrive at the same time,
- another where processes arrive at different times.

We'll create Gantt charts for both scenarios and calculate the turnaround time and waiting time for each process.

Scenario 1: Processes with Same Arrival Time

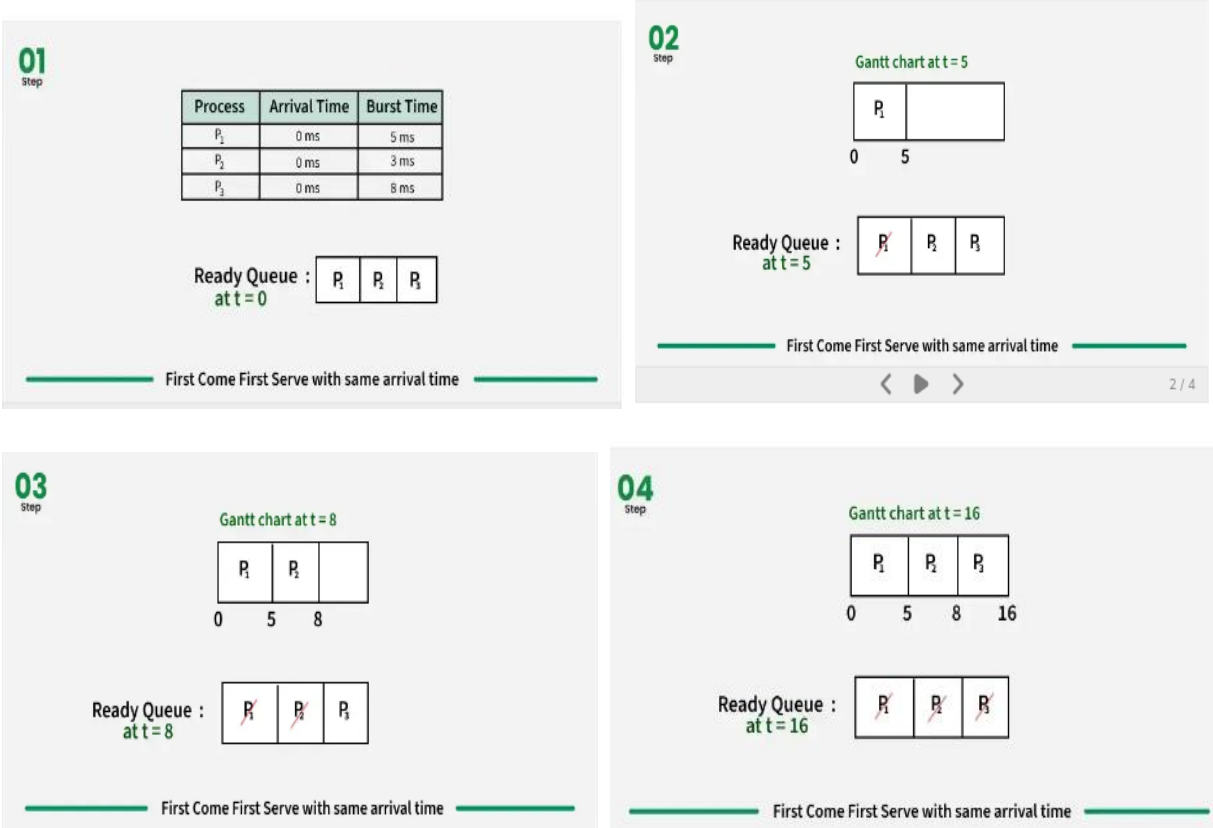
Consider the following table of arrival time and burst time for three processes P1, P2 and P3

Process	Arrival Time	Burst Time
p1	0	5
p2	0	3
p3	0	8

Step-by-Step Execution:

1. **P1** will start first and run for 5 units of time (from 0 to 5).
2. **P2** will start next and run for 3 units of time (from 5 to 8).
3. **P3** will run last, executing for 8 units (from 8 to 16).

Gant Chart:



Turnaround Time = Completion Time - Arrival Time
Waiting Time = Turnaround Time - Burst Time

AT : Arrival Time
BT : Burst Time or CPU Time
TAT : Turn Around Time
WT : Waiting Time

Processes	AT	BT	CT	TAT	WT
P1	0	5	5	5-0 = 5	5-5 = 0
P2	0	3	8	8-0 = 8	8-3 = 5
P3	0	8	16	16-0 = 16	16-8 = 8

- **Average Turn around time** = 9.67
- **Average waiting time** = 4.33

Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3

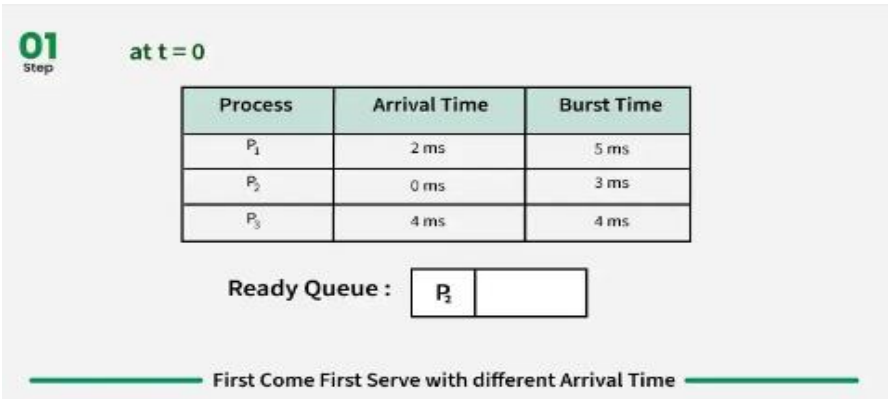
Process	Burst Time (BT)	Arrival Time (AT)
P1	**5 ms**	**2 ms**
P2	**3 ms**	**0 ms**
P3	**4 ms**	**4 ms**

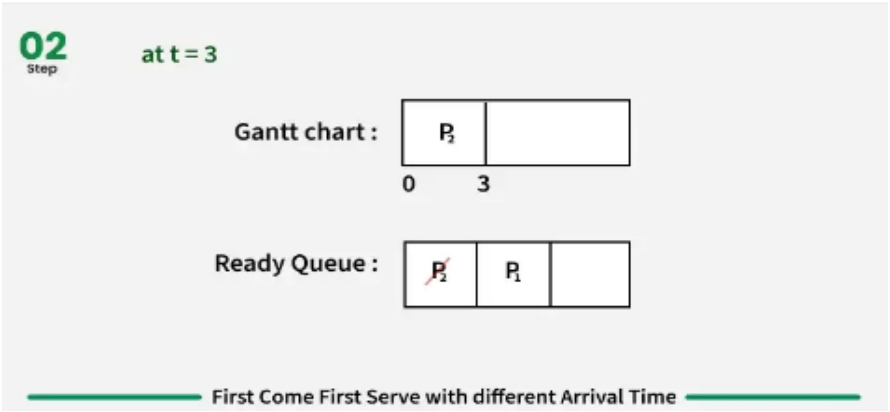
Step-by-Step Execution:

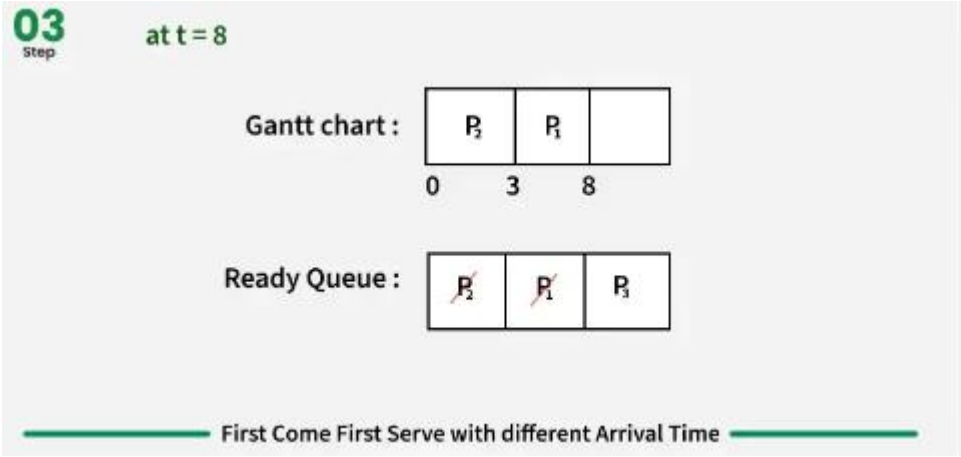
- P2 arrives at time 0 and runs for 3 units, so its completion time is: Completion Time of P2=0+3=3
- P1 arrives at time 2 but has to wait for P2 to finish. P1 starts at time 3 and runs for 5 units. Its completion time is: Completion Time of P1=3+5=8

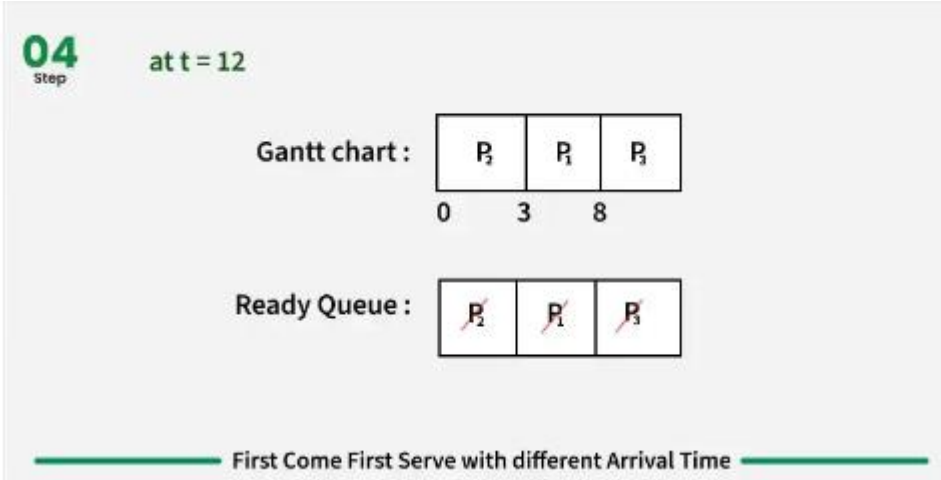
- P3 arrives at time 4 but has to wait for P1 to finish. P3 starts at time 8 and runs for 4 units. Its completion time is: Completion Time of P3=8+4=12

Gantt Chart:









Now, lets calculate average waiting time and turn around time:

Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P2	3 ms	3 ms	0 ms
P1	8 ms	6 ms	1 ms
P3	12 ms	8 ms	4 ms

- **Average Turnaround time = 5.67**
- **Average waiting time = 1.67**

Advantages of FCFS

- The simplest and basic form of CPU Scheduling algorithm
- Every process gets a chance to execute in the order of its arrival. This ensures that no process is arbitrarily prioritized over another.
- Easy to implement, it doesn't require complex data structures.
- Since processes are executed in the order they arrive, there's no risk of starvation
- It is well suited for batch systems where the longer time periods for each process are often acceptable.

Disadvantages of FCFS

- As it is a Non-preemptive CPU Scheduling Algorithm, FCFS can result in long waiting times, especially if a long process arrives before a shorter one. This is known as the convoy effect, where shorter processes are forced to wait behind longer processes, leading to inefficient execution.
- The average waiting time in the FCFS is much higher than in the others
- Since FCFS processes tasks in the order they arrive, short jobs may have to wait a long time if they arrive after longer tasks, which leads to poor performance in systems with a mix of long and short tasks.
- Processes that are at the end of the queue, have to wait longer to finish.
- It is not suitable for time-sharing operating systems where each process should get the same amount of CPU time.

SJF (Shortest Job First)

Shortest Job First (SJF) or Shortest Job Next (SJN) is a scheduling process that selects the waiting process with the smallest execution time to execute next. This scheduling method may or may not be preemptive. Significantly reduces the average waiting time for other processes waiting to be executed.

Characteristics of SJF Scheduling

- Shortest Job first has the advantage of having a minimum average waiting time among all operating system scheduling algorithms.
- It is associated with each task as a unit of time to complete.
- It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of ageing.

Example: Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

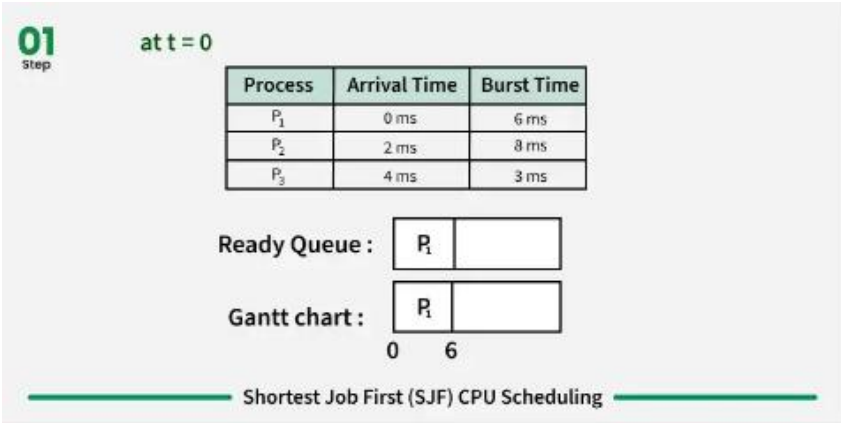
Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	2 ms
P3	3 ms	4 ms

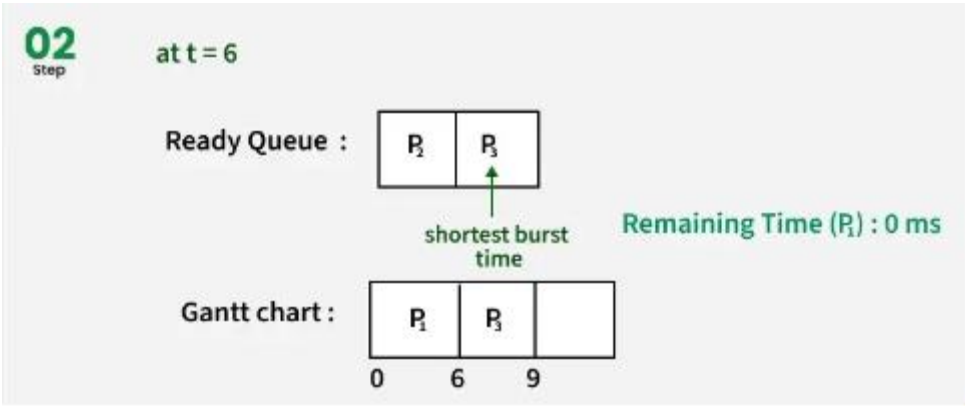
Step-by-Step Execution:

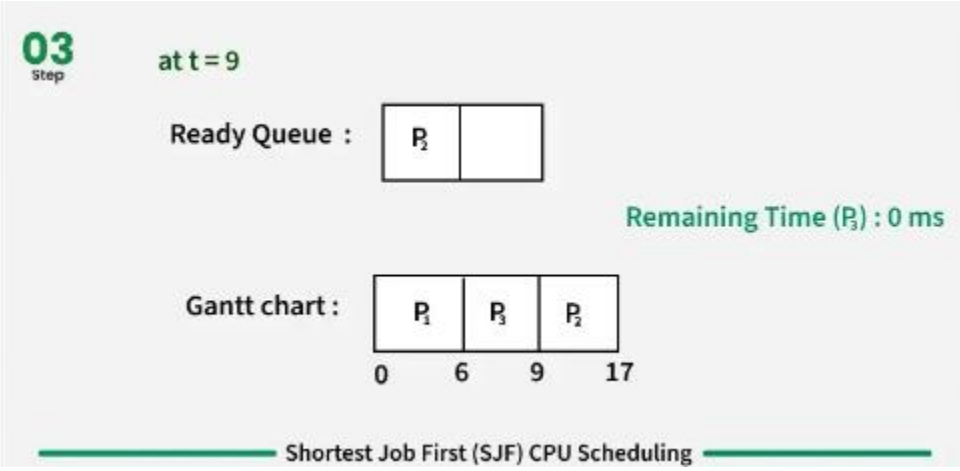
1. **Time 0-6 (P1)**: P1 runs for 6 ms (total time left: 0 ms)

2. **Time 6-9 (P3)**: P3 runs for 3 ms (total time left: 0 ms)

3. **Time 9-17 (P2)**: P2 runs for 8 ms (total time left: 0 ms)







As we know,

- ***Turn Around time*** *= Completion time - arrival time*
- ***Waiting Time*** *= Turn around time - burst time*

Now, lets calculate average waiting time and turn around time:

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	6	6-0 = 6	6-6 = 0
P2	2	8	17	17-2 = 15	15-8 = 7
P3	4	3	9	9-4 = 5	5-3 = 2

- ***Average Turn around time*** *= (6 + 15 + 5)/3 = 8.6 ms*
- ***Average waiting time*** *= (2 + 0 + 7)/3 = 9/3 = 3 ms*

Advantages of SJF Scheduling

- SJF is better than the First come first serve(FCFS) algorithm as it reduces the average waiting time.
- SJF is generally used for long term scheduling.
- It is suitable for the jobs running in batches, where run times are already known.
- SJF is probably optimal in terms of average Turn Around Time (TAT).

Disadvantages of SJF Scheduling

- SJF may cause very long turn-around times or starvation.
- In SJF job completion time must be known earlier.
- Many times it becomes complicated to predict the length of the upcoming CPU request.

SRTF (Shortest Remaining Time First)(Preemptive SJF)

The pre-emptive version of Shortest Job First (SJF) scheduling is called Shortest Remaining Time First (SRTF). In SRTF, the process with the least time left to finish is selected to run. The running process continues until it finishes or a new process with a shorter remaining time arrives, ensuring the fastest finishing process always gets priority.

Example of SJF Algorithm:

Scenario 1: Processes with Same Arrival Time

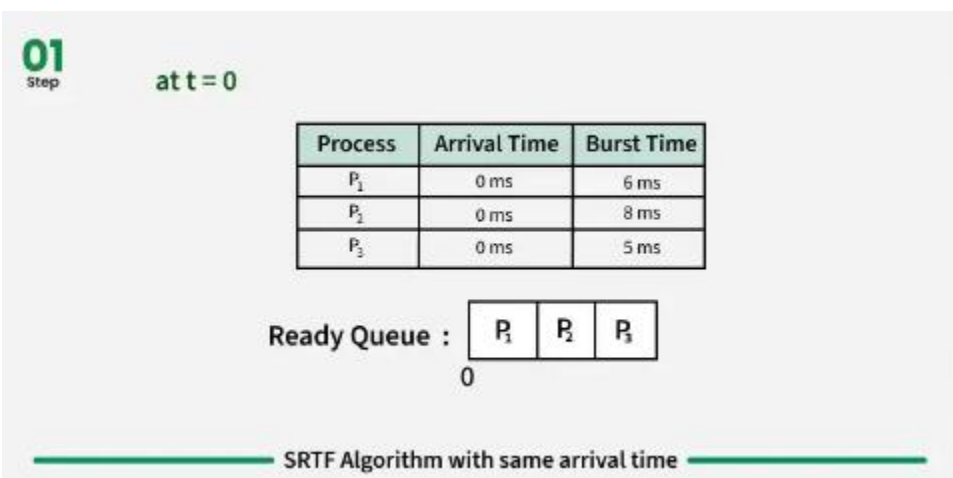
Example: Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

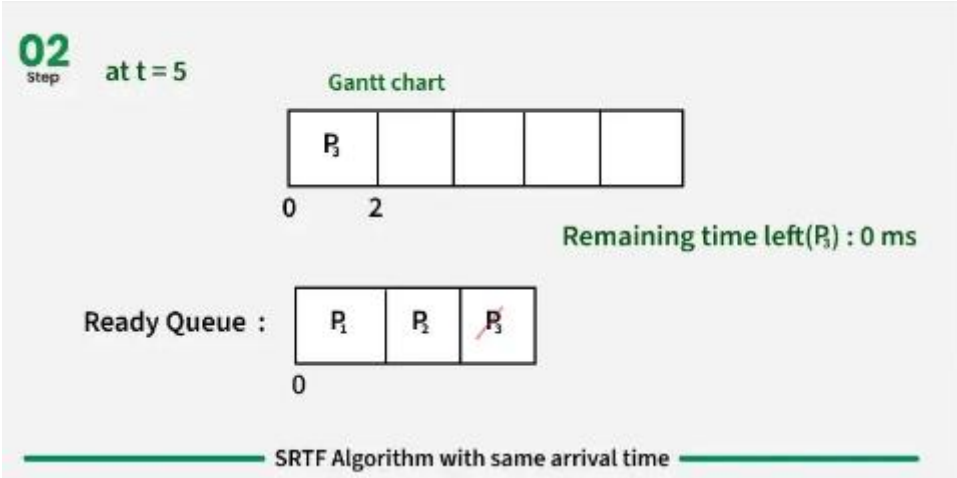
Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	0 ms
P3	5 ms	0 ms

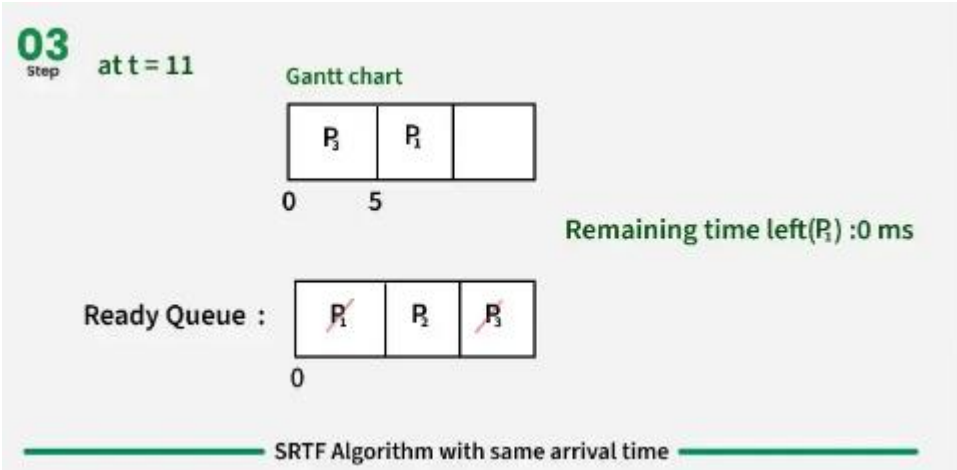
Step-by-Step Execution:

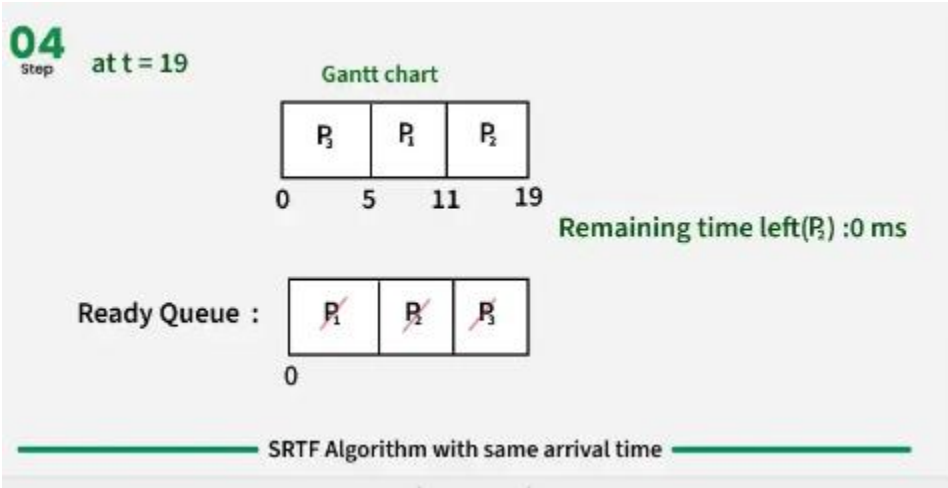
1. **Time 0-5 (P3):** P3 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left.
2. **Time 5-11 (P1):** P1 runs for 6 ms (total time left: 0 ms) as it has shortest remaining time left.
3. **Time 11-19 (P2):** P2 runs for 8 ms (total time left: 0 ms) as it has shortest remaining time left.

Gantt chart:









Now, lets calculate average [waiting time and turn around](#) time:

As we know,

- *Turn Around time = Completion time - arrival time*
- *Waiting Time = Turn around time - burst time*

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	11	11-0 = 11	11-6 = 5
P2	0	8	19	19-0 = 19	19-8 = 11
P3	0	5	5	5-0 = 5	5-5 = 0

Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	3 ms	1 ms
P3	7 ms	2 ms

Step-by-Step Execution:

1. Time 0-1 (P1): P1 runs for 1 ms (total time left: 5 ms) as it has shortest remaining time left.
2. Time 1-4 (P2): P2 runs for 3 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P2.
3. Time 4-9 (P1): P1 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P3.
4. Time 9-16 (P3): P3 runs for 7 ms (total time left: 0 ms) as it has shortest remaining time left.

01 Step

at t = 0

Process	Arrival Time	Burst Time
P_1	0 ms	6 ms
P_2	1 ms	3 ms
P_3	2 ms	7 ms

Ready Queue : | P_1 | |

0

SRTF Algorithm with different arrival time

02 Step

at t = 1

Process	Arrival Time	Burst Time
P_1	0 ms	~~6~~ 5 ms
P_2	1 ms	3 ms ← Shortest Remaining time
P_3	2 ms	7 ms

Gantt chart | P_1 | |

0 1

Ready Queue : | P_1 | P_2 | |

SRTF Algorithm with different arrival time

03 Step

at t = 4

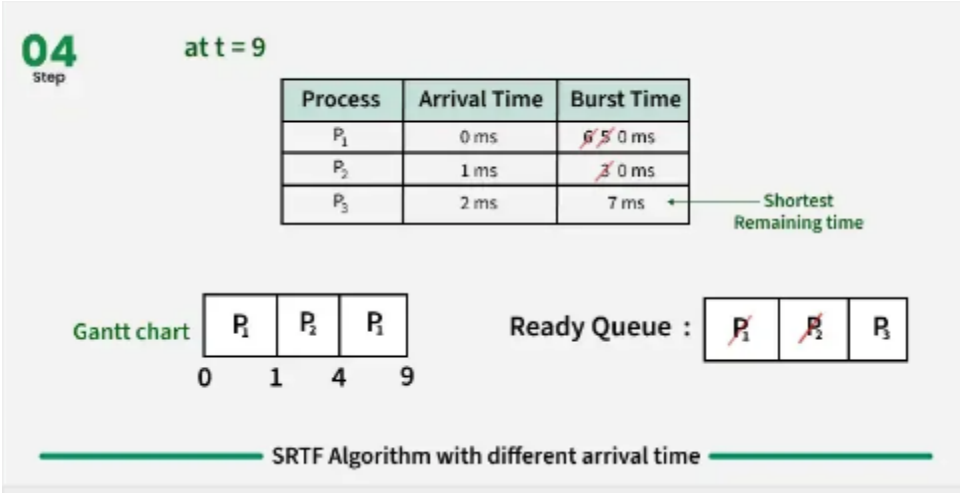
Process	Arrival Time	Burst Time
P_1	0 ms	~~6~~ 5 ms ← Shortest Remaining time
P_2	1 ms	~~3~~ 0 ms
P_3	2 ms	7 ms

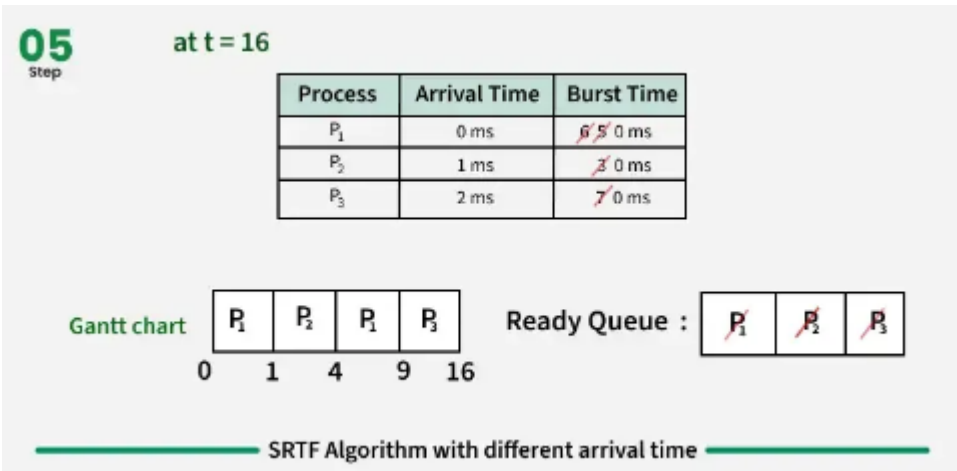
Gantt chart | P_1 | P_2 | |

0 1 4

Ready Queue : | P_1 | ~~P_2~~ | P_3 |

SRTF Algorithm with different arrival time





Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	9	9-0 = 9	9-6 = 3
P2	1	3	4	4-1 = 3	3-3 = 0
P3	2	7	16	16-2 = 14	14-7 = 7

- *Average Turn around time = (9 + 14 + 3)/3 = 8.6 ms*
- *Average waiting time = (3 + 0 + 7)/3 = 10/3 = 3.33 ms*

Advantages of SRTF Scheduling

- Minimizes average waiting time by always selecting the process with the shortest remaining time.
- Short processes finish quickly, improving system responsiveness and throughput.
- Helps time-critical tasks get CPU attention earlier than longer jobs.

Disadvantages of SRTF Scheduling

- Long processes may suffer starvation if short jobs keep arriving continuously.
- Accurate prediction of CPU burst time is difficult and often unreliable.
- Frequent context switching increases overhead and can reduce overall performance.

RR (Round Robin)

Round Robin Scheduling is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention. It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.

The primary goal of this scheduling method is to ensure that all processes are given an equal opportunity to execute, promoting fairness among tasks.

- **Process Arrival:** Processes enter the system and are placed in a queue.
- **Time Allocation:** Each process is given a certain amount of CPU time, called a quantum.
- **Execution:** The process uses the CPU for the allocated time.
- **Rotation:** If the process completes within the time, it leaves the system. If not, it goes back to the end of the queue.
- **Repeat:** The CPU continues to cycle through the queue until all processes are completed.

Example:

To understand the Round Robin Scheduling algorithm, let’s consider the following two scenarios:

Scenario 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2 ms**

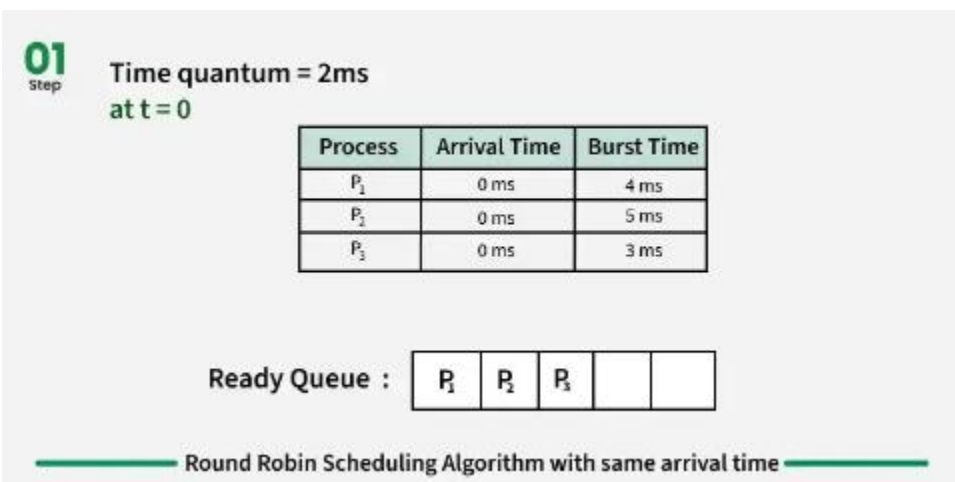
Process	Burst Time	Arrival Time
P1	4 ms	0 ms

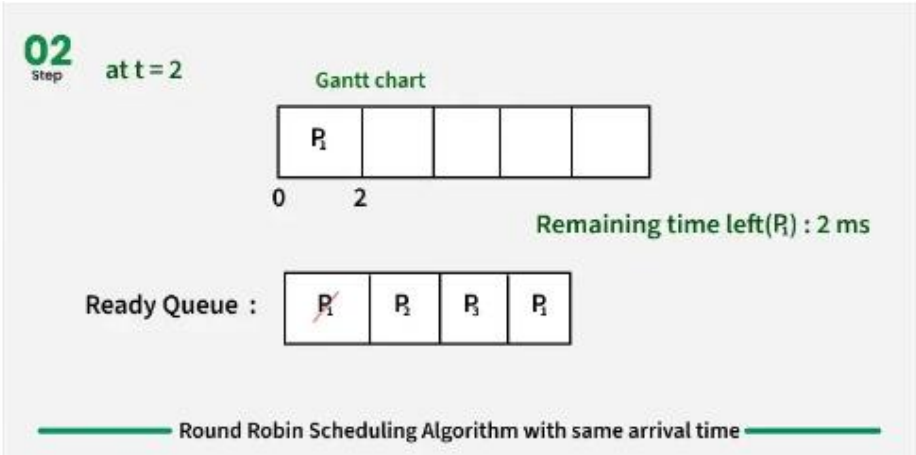
Process	Burst Time	Arrival Time
P2	5 ms	0 ms
P3	3 ms	0 ms

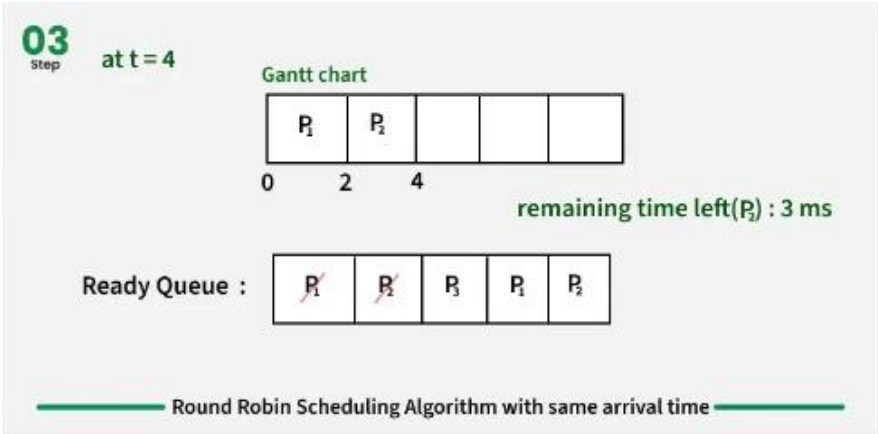
Step-by-Step Execution:

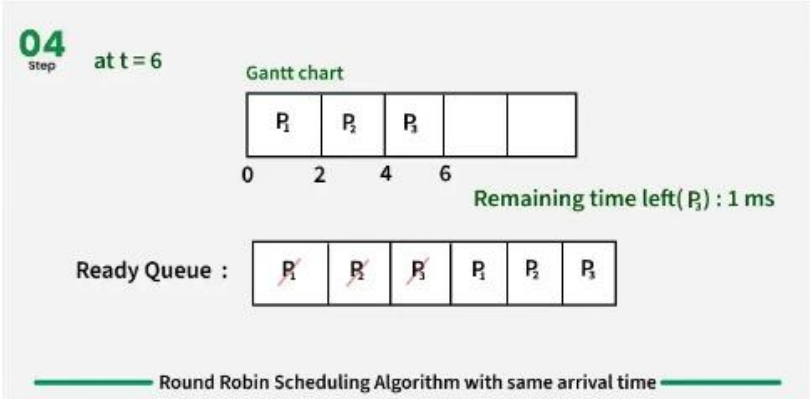
1. **Time 0-2 (P1)**: P1 runs for 2 ms (total time left: 2 ms).
2. **Time 2-4 (P2)**: P2 runs for 2 ms (total time left: 3 ms).
3. **Time 4-6 (P3)**: P3 runs for 2 ms (total time left: 1 ms).
4. **Time 6-8 (P1)**: P1 finishes its last 2 ms.
5. **Time 8-10 (P2)**: P2 runs for another 2 ms (total time left: 1 ms).
6. **Time 10-11 (P3)**: P3 finishes its last 1 ms.
7. **Time 11-12 (P2)**: P2 finishes its last 1 ms.

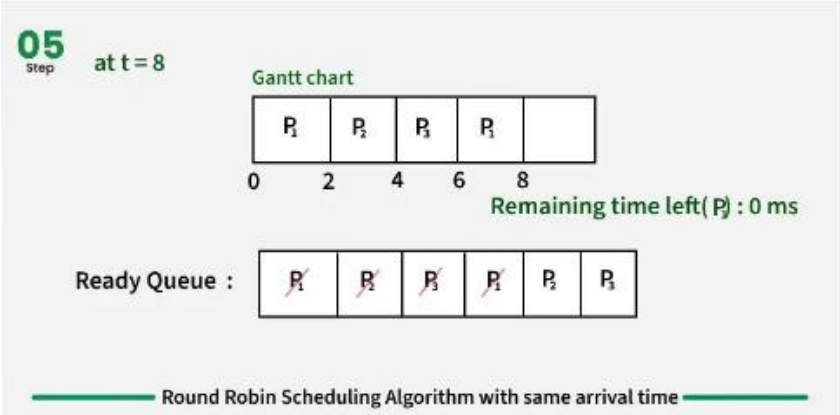
Gantt Chart:











Now, lets calculate average waiting time and turn around time:

- ***Turnaround Time*** *= Completion Time - Arrival Time*
- ***Waiting Time*** *= Turnaround Time - Burst Time*

Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- **Average Turn around time** = (8 + 12 + 11)/3 = 31/3 = 10.33 ms
- **Average waiting time** = (4 + 7 + 8)/3 = 19/3 = 6.33 ms

Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

Step-by-Step Execution:

1. Time 0-2 (P1 Executes):

- P1 starts execution as it arrives at 0 ms.
- Runs for 2 ms; remaining burst time = 5 - 2 = 3 ms.
- **Ready Queue:** [P1].

2. Time 2-4 (P1 Executes Again):

- P1 continues execution since no other process has arrived yet.
- Runs for 2 ms; remaining burst time = 3 - 2 = 1 ms.

- P2 arrive at 4 ms.
- **Ready Queue:** [P2, P1].

3. Time 4-6 (P2 Executes):

- P2 starts execution as it arrives at 4 ms.
- Runs for 2 ms; remaining burst time = 2 - 2 = 0 ms.
- P3 arrive at 5ms
- **Ready Queue:** [P1, P3].

4. Time 6-7 (P1 Executes):

- P1 starts execution.
- Runs for 1 ms; remaining burst time = 1 - 1 = 0 ms.
- **Ready Queue:** [P3].

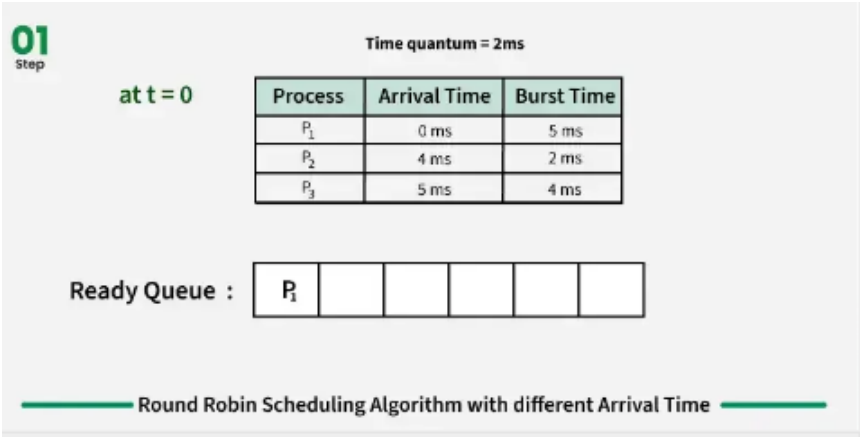
5. Time 7-9 (P3 Executes):

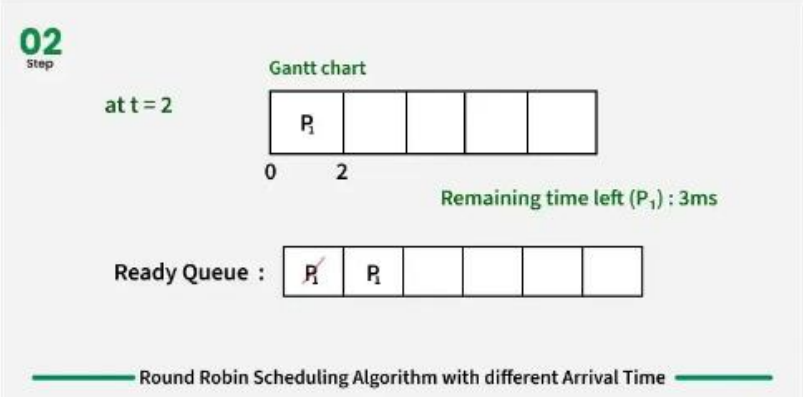
- P3 starts execution.
- Remaining burst time = 4 - 2 = 2 ms.
- **Ready Queue:** [P3].

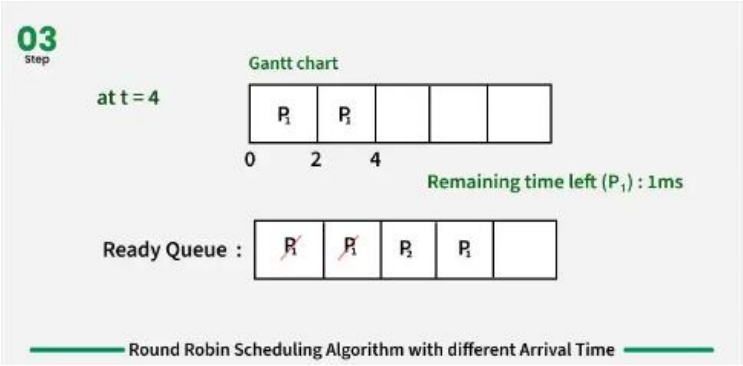
6. Time 9-11 (P3 Executes Again):

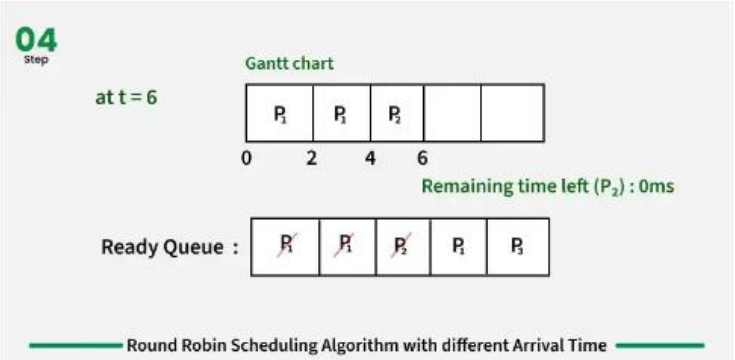
- P3 resumes execution and runs for 2 ms and complete its execution
- Remaining burst time = 2 - 2 = 0 ms.
- **Ready Queue:** [].

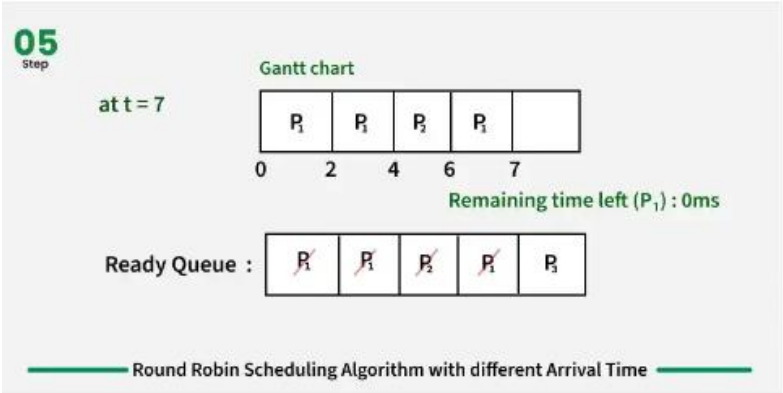
Gantt Chart:

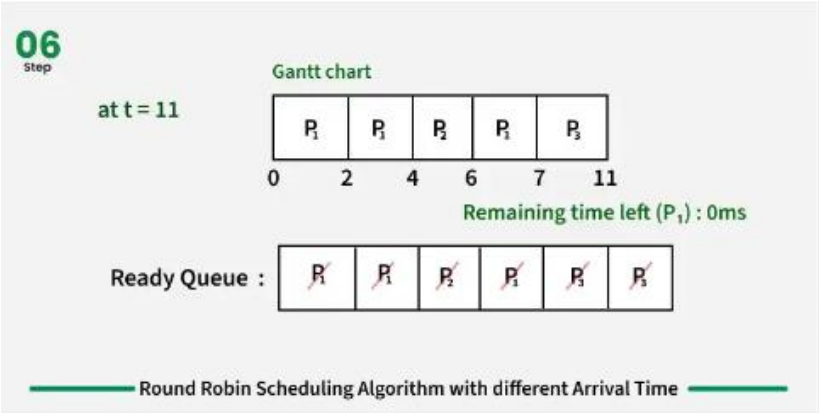



02
Step
Gantt chart
at t = 2
P1
0
2
Remaining time left (P1) : 3ms
Ready Queue :
P1
P1
Round Robin Scheduling Algorithm with different Arrival Time


03
Step
Gantt chart
at t = 4
P1
P1
0
2
4
Remaining time left (P1) : 1ms
Ready Queue :
P1
P1
P2
P1
Round Robin Scheduling Algorithm with different Arrival Time


04
Step
Gantt chart
at t = 6
P1
P1
P2
0
2
4
6
Remaining time left (P2) : 0ms
Ready Queue :
P1
P1
P2
P1
P3
Round Robin Scheduling Algorithm with different Arrival Time


05
Step
Gantt chart
at t = 7
P1
P1
P2
P1
0
2
4
6
7
Remaining time left (P1) : 0ms
Ready Queue :
P1
P1
P2
P1
P3
Round Robin Scheduling Algorithm with different Arrival Time



Now, lets calculate average waiting time and turn around time:

Process	**Completion Time (CT)**	**Turnaround Time (TAT = CT - AT)**	**Waiting Time (WT = TAT - BT)**
P1	7 ms	7 ms	2 ms
P2	6 ms	2 ms	0 ms
P3	11 ms	6 ms	2 ms

- **Average Turn around time** =7+2+6/3=15/3=5ms
- **Average waiting time** = 2+0+2/3=1.33ms